

## Highlights

### **R4Tun: LLM-guided adaptive segmental tunnel lining segmentation in point clouds**

Xinghui Tao, Guangming Wang, Jelena Ninić, Brian Sheil

- Introduce R4Tun that tunes tunnel lining segmentation when conditions shift.
- Stage agents adapt expert pipeline parameters using memory (m), state (s), and knowledge (k).
- Across 30 tunnel subsets,  $m + s + k$  raised overall mean mIoU from 0.150 to 0.316–0.342.
- Ablations suggest state provides the main increment; knowledge adds a smaller complex-tunnel gain.
- R4Tun supports controlled, cross-LLM, zero-label adaptation of an expert pipeline in the tested setting.

# R4Tun: LLM-guided adaptive segmental tunnel lining segmentation in point clouds

Xinghui Tao<sup>a</sup>, Guangming Wang<sup>a,\*</sup>, Jelena Ninić<sup>b</sup> and Brian Sheil<sup>a</sup>

<sup>a</sup> *Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, United Kingdom*

<sup>b</sup> *Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, United Kingdom*

## ARTICLE INFO

### Keywords:

Segmental tunnel lining  
Point cloud segmentation  
Tunnel inspection  
Large language models  
Parameter adaptation  
Multi-agent systems

## ABSTRACT

Automated inspection of segmental tunnel linings requires adaptive segmentation from 3D point clouds, yet expert-tuned pipelines often degrade when tunnel conditions vary. This paper presents R4Tun, a large language model (LLM)-driven adaptation framework that extends an expert-designed pipeline (SAM4Tun) with bounded parameter tuning informed by structured context: memory ( $m$ ), state ( $s$ ), and knowledge ( $k$ ). Evaluated on 30 selected Seg2Tunnel subsets (13 regular, 17 complex) across three LLMs, the full  $m + s + k$  design raised mean Intersection-over-Union (mIoU) from 0.15 to 0.32–0.34 and overall accuracy (OA) from 0.41 to 0.52–0.55 relative to the static SAM4Tun baseline. On complex tunnels, mIoU rose from 0.04 to 0.18–0.19. Across 270 (30 tunnels  $\times$  3 different LLMs  $\times$  3 context settings) runs, the LLMs showed similar parameter-adjustment trends and consistently adjusted a shared set of critical parameters. **These results support R4Tun as a controlled, cross-LLM, zero-label adaptation mechanism in the tested SAM4Tun–Seg2Tunnel setting; they do not establish a deployable autonomous inspection system or a new absolute-accuracy state of the art.**

## 1. Introduction

Automated inspection of segmental tunnel linings is required to support structural health assessment and long-term operational safety, given the scale, frequency, and access constraints of modern tunnel networks [2]. In practice, engineers routinely encounter projects in which tunnel geometry, lining properties, and data acquisition settings vary [46]. Furthermore, although modern laser scanning enables high-fidelity tunnel capture, the resulting measurements often contain mixed structural elements, occlusions, and noise, making direct extraction of lining components challenging [21, 39]. Consequently, segmentation methods need to be adaptive enough to handle this complexity and variability [29, 40].

Existing tunnel point-cloud segmentation methods are grouped into three broad categories: feature engineering, deep learning and foundation-model pipelines (see Fig. 1 and Section 2). Feature engineering rules are interpretable but lack robustness under changed conditions [46]. Supervised deep-learning models generalise through data but require large labelled datasets and periodic retraining, while lacking auditability [11, 7, 41]. Foundation-model pipelines bypass annotation by using models pre-trained on large datasets [3, 22], but remain sensitive to expert-tuned processing parameters. Existing tunnel-segmentation approaches have not yet demonstrated, in a single workflow, both

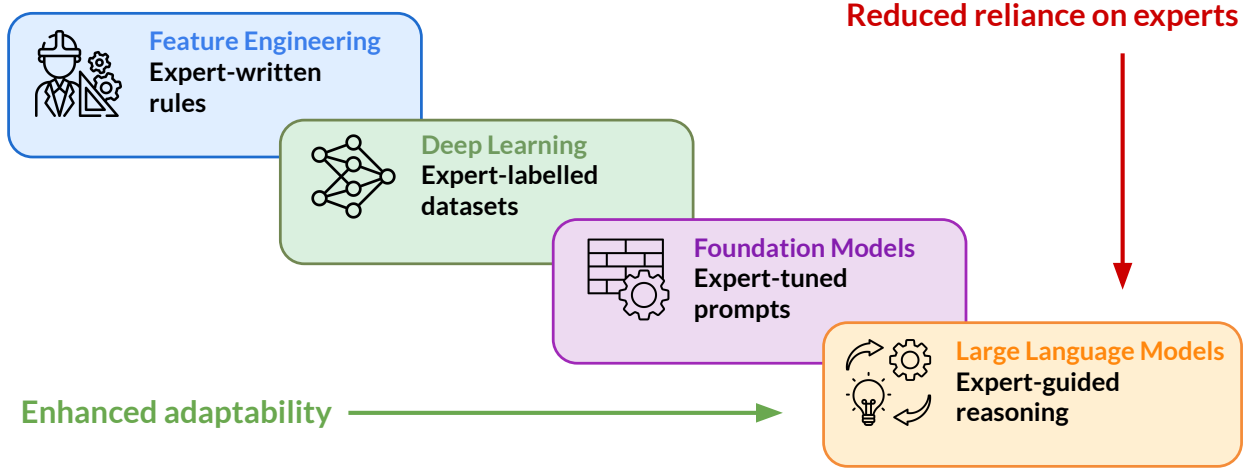
robust adaptation to changed conditions and an auditable adaptation process. Among foundation-model pipelines, SAM4Tun [51] combines point-cloud preprocessing with SAM-based prompting [22] and has shown strong performance under expert tuning. However, the pipeline is highly sensitive to its many processing parameters. When tunnel geometry or scanning conditions depart from the tuning reference, performance can degrade, and identifying which parameters need adjustment can require repeated expert intervention.

Given that recent LLMs can follow multi-step instructions over structured inputs, parameter adaptation can be formulated as a diagnostic task in which the model receives context and proposes bounded adjustments. LLMs have been applied to engineering workflows including chemical synthesis planning [4], multi-agent coordination for complex project tasks [36, 19, 8], and manufacturing decision support [16]. However, these studies do not address parameter adaptation in infrastructure inspection pipelines. In this study, we propose R4Tun, an LLM-guided adaptation system that extends the SAM4Tun pipeline by adjusting stage parameters in response to changing tunnel conditions. The framework provides each agent with structured context  $m + s + k$  (memory, state, and knowledge), from which it produces bounded parameter updates via stepwise prompting. We evaluate whether this contextual information improves segmentation adaptability across both regular and complex tunnels, and whether the resulting adaptation behaviour is consistent across LLMs. To isolate the contribution of the LLM beyond deterministic tuning, we also benchmark R4Tun

\*Corresponding author

 gw462@cam.ac.uk (G. Wang)

ORCID(s):



**Figure 1:** Automated segmentation approaches arranged by their adaptation mechanisms. Expert-guided, LLM-driven methods can extend foundation-model pipelines by adding reasoning-guided parameter adaptation conditioned on structured context and encoded expert knowledge at deployment. In this work, we test one such mechanism (R4Tun) under controlled, cross-LLM, zero-label settings while keeping the underlying SAM4Tun operators fixed. R4Tun reuses expert-defined reference parameters, parameter bounds, and knowledge documents, so it reduces deployment-stage retuning within a predefined pipeline rather than removing expert knowledge from the workflow.

against a rule-based adaptation derived from the same per-stage knowledge documents.

We position R4Tun as a mechanism contribution rather than as a deployable final-inspection system. The strength of the framework lies in integrating feature-engineered preprocessing, a foundation segmentation model, and LLM-guided bounded parameter adaptation within a single auditable workflow. The object of study is whether structured context helps an LLM re-parameterise a frozen expert pipeline under controlled, auditable, label-free conditions; the absolute segmentation accuracy remains bounded by the fixed SAM4Tun substrate and by the single expert reference used in this experiment.

This paper makes the following contributions:

1. A framework design in which LLM agents adapt the parameters of the SAM4Tun pipeline using structured context  $m + s + k$ : memory of the reference configuration, state from intermediate pipeline outputs, and a knowledge document of tunnel category-specific configuration shared across all tunnels.
2. Empirical evidence from cumulative ablation across 30 tunnels suggesting that intermediate pipeline state is an important contributor to adaptation, while the knowledge component adds a smaller increment concentrated on the complex category.
3. Cross-LLM evidence showing that three different LLMs (Opus-4.6, GPT-5.4, Gemini-3-Flash) produce overlapping effect ranges and adjust a consistent subset of critical parameters under the same prompt structure.

4. Evidence that deterministic, rule-based adaptation explains part, but not all, of the observed mIoU increase, which is consistent with an additional contribution from the LLM-based adaptation process in the tested setting.

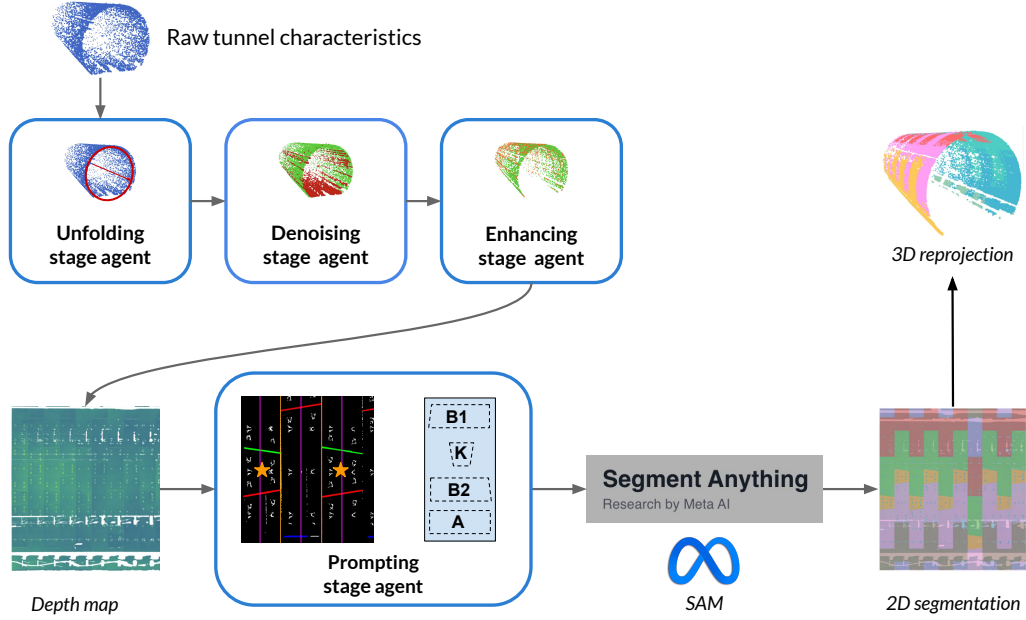
The rest of this paper is organised as follows. Section 2 reviews related work. Section 3 presents methodology, including the R4Tun architecture, dataset, experimental design, and evaluation. Section 4 reports results. Section 5 discusses findings, implications, and limitations. Section 6 concludes.

## 2. Related work

This section reviews related work on tunnel point-cloud segmentation. It first contrasts feature-engineered approaches with supervised deep-learning methods, then reviews foundation-model pipelines that reduce annotation demand but remain sensitive to pre-processing and prompting choices. Finally, it discusses LLM-based reasoning as a mechanism for bounded parameter adaptation, motivating the gap addressed by R4Tun.

### 2.1. Feature engineering versus deep learning

Tunnel point-cloud segmentation has advanced through two generations of methods, which trade off between auditability and adaptability. Feature-engineering approaches encode domain knowledge into deterministic geometric rules: centreline fitting via RANSAC [14], density-based surface clustering [13], and point-sampled surface processing (e.g., simplification and resampling) [34]. Those methods remain



**Figure 2:** The R4Tun multi-agent architecture for tunnel segmentation. Four stage agents (unfolding, denoising, enhancing, and segmenting) adapt their parameters through LLM reasoning to generate segmentation.

widely used in safety-critical inspection because their logic is explicit and auditable [46]. However, each rule embeds assumptions about tunnel geometry and point-cloud characteristics (diameter, ring spacing, joint pattern, point density), and changing any of these requires manual reconfiguration by a domain expert. Supervised deep-learning methods, such as PointNet++ [35], RandLA-Net [20], Mask3D [38], and TD3D [24], learn features directly from annotated point clouds and can generalise across similar conditions present in the training set. Recent work has also extended tunnel and infrastructure segmentation through transformer-based point-cloud models, geometric tunnel extraction, and image-based defect segmentation [43, 44, 50, 25]. Yet they require large labelled tunnel datasets that remain scarce [41], demand retraining for new tunnel conditions, and provide no mechanism for an engineer to trace or override a segmentation decision [7].

The need for reliable automated interpretation is broader than component segmentation alone. Recent infrastructure-inspection studies include non-contact road-surface inspection, damage segmentation for heritage structures, scaled model tests of shield-tunnel damage mechanisms caused by cavities behind the lining, and GPR-based transfer learning for hidden tunnel-lining defects [54, 52, 55, 53]. These studies reinforce the demand for scalable inspection analytics, but they address different sensing modalities, defect targets, or structural-damage questions rather than bounded parameter adaptation of a tunnel point-cloud segmentation pipeline.

Therefore, for inspection operators, deploying to a new tunnel type requires either re-engineering rules, collecting new training data, or accepting degraded performance without a systematic diagnostic mechanism.

## 2.2. Foundation-model pipelines

To the best of our knowledge, direct 3D point-cloud segmentation of infrastructure at tunnel scene scale remains an open challenge for current foundation models. Existing 3D foundation models target common object recognition on curated datasets and do not transfer to large, noisy tunnel point clouds [5, 27, 18, 49]. As a result, practical tunnel pipelines often project 3D data into 2D representations to use mature vision foundation models pre-trained on large-scale image datasets, thereby bypassing the annotation bottleneck [6, 15]. SAM [22] performs class-agnostic segmentation from spatial prompts without task-specific training, and extensions such as SAM2 [37] for temporal consistency and SEEM [58] for natural-language-guided segmentation are broadening prompt-based segmentation further. These models have been adopted in infrastructure contexts including crack detection [17] and Scan-to-BIM workflows [42, 33].

For tunnel linings, SAM4Tun [51] combines geometric preprocessing (unfolding, denoising, enhancing) with foundation-model 2D segmentation, achieving strong segmentation without training data. However, this pipeline depends on numerous stage-specific parameters that encode assumptions about the reference tunnel's diameter, ring length, point density, and joint patterns. When a new tunnel departs from the reference, these parameters become misspecified and

**Table 1**

Contextual positioning of recent tunnel and infrastructure segmentation approaches. The rows summarise task setting and adaptation mode rather than a direct accuracy leaderboard, because the studies differ in input modality, labels, deployment tuning, and evaluation protocol.

| Study             | Input / task                          | Labelled training    | Deployment adaptation | Reported role           |
|-------------------|---------------------------------------|----------------------|-----------------------|-------------------------|
| Seg2Tunnel [26]   | 3D point-cloud component segmentation | Yes                  | Learned model         | Supervised benchmark    |
| SAM4Tun [51]      | 3D-to-2D foundation-model pipeline    | No                   | Expert per scenario   | Strong no-training segm |
| SerialFormer [43] | 3D metro-tunnel semantic segmentation | Yes                  | Learned model         | Transformer-based geom  |
| GuSAC/TMatch [44] | Tunnel point-cloud extraction         | No                   | Geometric rules       | Parameter-light tunnel  |
| YOLOv8-GSD [50]   | Tunnel crack image segmentation       | Yes                  | Learned model         | Defect segmentation an  |
| R4Tun (this work) | SAM4Tun parameter adaptation          | No (evaluation only) | LLM-bounded tuning    | Tests whether structur  |

the pipeline can degrade without indicating which parameters fail or why. This sensitivity is not unique to SAM4Tun: most multi-stage pipelines that chain domain-specific preprocessing with a foundation model tend to inherit it. While foundation models have reduced reliance on labelled data, they have made the pipeline's dependence on hand-tuned parameters more explicit, without removing the need for expert parameter tuning.

### 2.3. LLM reasoning

Recent LLM advances have introduced explicit reasoning capabilities relevant to the parameter-adaptation problem. Reasoning-oriented training [32, 10, 9, 47, 30, 31] produces models capable of decomposing complex problems and performing prompted consistency checks. Chain-of-thought (CoT) reasoning [45, 23] enables step-by-step logical traces. Multi-agent architectures coordinate specialised roles through shared context [19, 36, 8, 57, 56], and context engineering influences reasoning quality through the careful design of information provided to models [48, 28, 1].

In tunnelling segmentation, however, no prior work has applied LLM-based reasoning to a challenge in expert-designed pipelines: re-tuning a parameter-sensitive system to new conditions while preserving the engineer's ability to inspect and override each decision. We therefore introduce an LLM-based reasoning framework that retunes parameter-sensitive pipelines per tunnel using expert-guided context and a logged rationale for each proposed change. Meanwhile, whether LLM reasoning produces gains beyond what hand-coded rules can deliver from the same knowledge text remains an open question. Our experimental design therefore tests this directly by including a deterministic rule-based adaptation of those documents as a non-LLM control.

## 3. Methodology

This section describes the task, the fixed SAM4Tun pipeline, the R4Tun adaptation layer using structured context, and finally the experimental design used to evaluate bounded parameter adaptation, including the

dataset, baselines, ablation settings, metrics, and sensitivity analysis.

### 3.1. Task definition

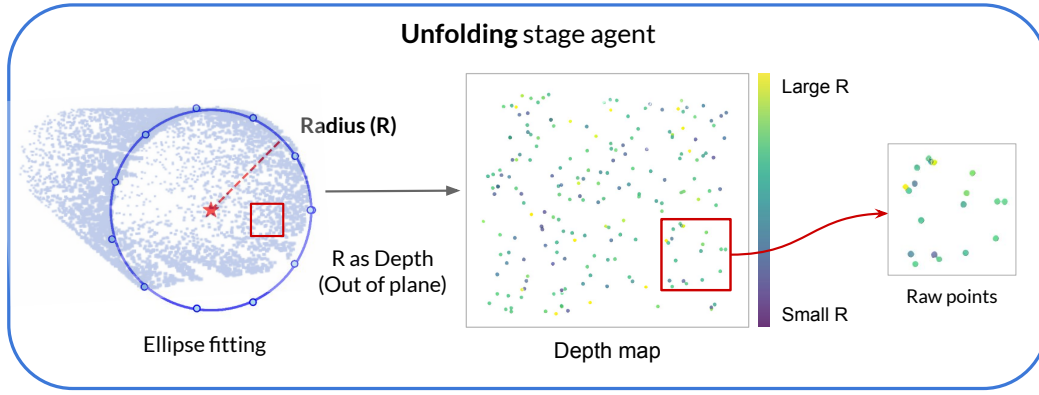
The task is semantic segmentation of segmental tunnel linings from terrestrial laser scanning (TLS) point clouds. Given a raw point cloud of a tunnel section, the goal is to assign each point a structural label: background, key block (K), base blocks (B1, B2), and adjacent blocks (A1–A3), with an additional A4 class for 7-segment complex tunnels. The unit of analysis is a tunnel subset spanning multiple rings selected from the Seg2Tunnel benchmark.

A key limitation is that the fixed configuration of the open-source SAM4Tun achieves a mean mIoU of 0.29 on regular tunnels, but only 0.04 on complex tunnels whose geometry departs from the tuning reference. As noted above, the pipeline provides no built-in signal indicating which parameters become misspecified.

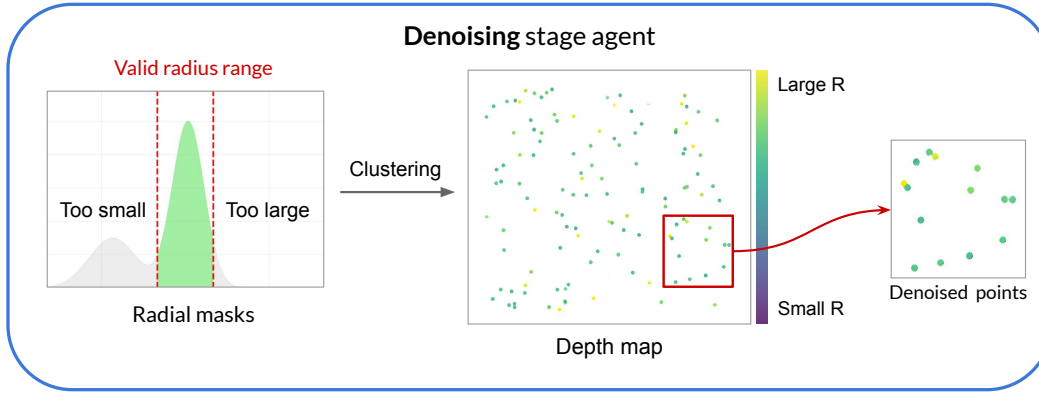
### 3.2. R4Tun

R4Tun supplements the SAM4Tun pipeline [51] with an LLM-guided adaptation layer that adjusts parameters per tunnel without modifying the underlying algorithms (Fig. 2). Given a new tunnel point cloud, a characteriser first extracts raw geometric properties (diameter, density, coordinate ranges; full field list in Appendix B). Each of the four stages (unfolding, denoising, enhancing, and segmenting) has a dedicated LLM agent that adapts its parameters. In the segmenting stage, the adapted prompts are passed to SAM, and the resulting 2D masks are reprojected onto the 3D point cloud without further LLM intervention.

**Stage 1-Unfolding** (Fig. 3) establishes a cylindrical coordinate system for the tunnel. It slices the point cloud at regular longitudinal intervals, fits an ellipse to each cross-section via RANSAC [14], and selects the model with the highest inlier support. The per-slice ellipse centres are interpolated into a smooth centreline that defines the cylindrical frame  $(r, \theta, h)$ . The 3D tunnel surface is then projected into a 2D panoramic image in which each pixel encodes the radial distance to the centreline, producing a stable depth representation for downstream filtering and segmentation.



**Figure 3:** Stage 1-Unfolding. Left: fitted tunnel cross-section used to determine the reference centre and radius for cylindrical unfolding. Centre: 2D panoramic depth map derived from the 3D point cloud. Right: zoomed region of the unfolded map showing radial-distance encoding.



**Figure 4:** Stage 2-Denoising. Left: radial-distance distribution used to filter points outside the expected tunnel lining surface. Centre: unfolded 2D depth map after filtering, showing the cleaned point distribution. Right: zoomed region of the denoised map.

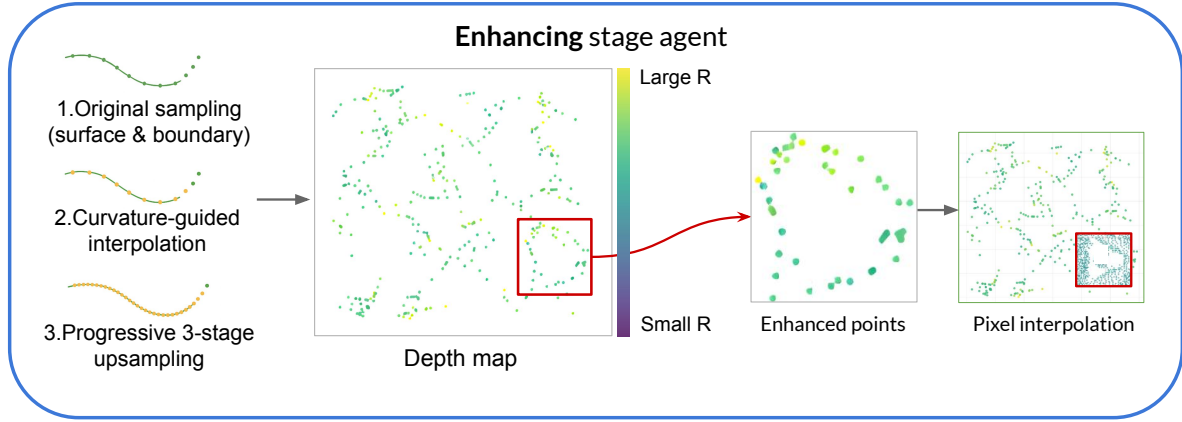
**Stage 2-Denoising** (Fig. 4) removes non-structural artefacts (rails, cables, and scattered points) that would otherwise interfere with segmentation. The algorithm applies grid-based radial-density filtering inspired by the DBSCAN clustering principle [13], grouping points by local density in cylindrical coordinates. Dense, connected regions are retained as tunnel surfaces while isolated points are rejected as noise. A pair of radial masks ( $R_{low}$ ,  $R_{high}$ ) constrains filtering to the expected tunnel radius, preserving joint boundaries and ring edges while discarding outliers beyond the lining surface. The result is a clean, continuous surface representation suitable for curvature analysis and interpolation in the next stage.

**Stage 3-Enhancing** (Fig. 5) improves geometric continuity and surface completeness before projection into the 2D depth map. Local curvature is estimated on neighbourhood points to guide point insertion: new points are placed between neighbours whose curvature difference is small (smooth panel regions), while high-curvature areas (joint boundaries and ring edges) are preserved intact. A three-stage progressive upsampling scheme inserts additional points at progressively finer

resolutions to maintain surface continuity without blurring structural boundaries. Pixel-level interpolation then refines outlier points at joint locations. The enhanced surface is projected into a panoramic depth map that balances smooth panel regions with sharply defined joint boundaries.

**Stage 4-Segmenting** (Fig. 6) combines boundary detection and SAM segmentation into a single workflow. First, a Hough-transform-based detector [12] extracts linear features from the enhanced depth map that correspond to ring joint boundaries. Detected lines are filtered by orientation (oblique, horizontal, vertical) with separate Hough thresholds and merged when closer than a distance tolerance. The confirmed boundaries are then used to construct template-based prompts for key, base, and adjacent segments, which are passed to SAM [22]. SAM produces 2D segment masks on the panoramic image, which are reprojected into 3D point labels via the stored pixel-to-point mapping from Stage 1.

The pipeline contains 81 tunable processing parameters, spanning geometric defaults, filtering and interpolation settings, and boundary-detection thresholds. All parameters were expert-tuned on a reference tunnel



**Figure 5:** Stage 3-Enhancing. Left: curvature-guided surface and boundary interpolation along the tunnel lining, showing original sampling, curvature-guided insertion, and progressive three-stage upsampling. Centre: unfolded 2D depth map after enhancement, where inserted points improve geometric continuity; the red box highlights a local region of interest. Right: zoomed views of the highlighted region, showing added points and pixel-level interpolation. **The panels show sequential steps in one enhancement workflow, not alternative interpolation choices.**

(diameter 5.60 m, density 2,466 pts/m<sup>3</sup>, mIoU = 0.531); full parameter tables are provided in Appendix A. In this study, the underlying pipeline stages remain fixed across all conditions; only the parameter values change. The SAM4Tun baseline applies this single expert-tuned configuration uniformly to all 30 tunnels without per-tunnel adaptation, serving as the static baseline against which LLM-guided adaptation is measured.

### 3.3. Agent design

Each of the four stage agents is a self-contained unit comprising (i) a *context* (Section 3.3.1; Fig. 7a) and (ii) a Python *analyst* that constructs the full LLM prompt and parses the returned JSON. For each agent-driven stage, the analyst assembles a prompt from the current context level, calls one of the selected LLM APIs (Opus-4.6, GPT-5.4, or Gemini-3-Flash) to follow a five-step CoT protocol (Section 3.3.2; Fig. 7b): referencing, diagnostic inspection, parameter adaptation, validation, and JSON output. Each agent returns a single schema-conformant parameter JSON, which the corresponding pipeline stage executes unchanged. A characteriser plugin then extracts statistics from the stage output, updating the cumulative state available to subsequent stages. Each stage's parameters are adapted from the expert-tuned reference configuration together with this cumulative state; the stage scripts and evaluation code are identical across all ablation conditions, with only the parameter JSONs changing. Because every parameter change is logged alongside a generated textual rationale, engineers can review and, where necessary, override adaptation decisions. A worked example is provided in Appendix D. After the final segmentation, per-point labels are evaluated against ground truth using mean Intersection over Union (mIoU) as the primary metric.

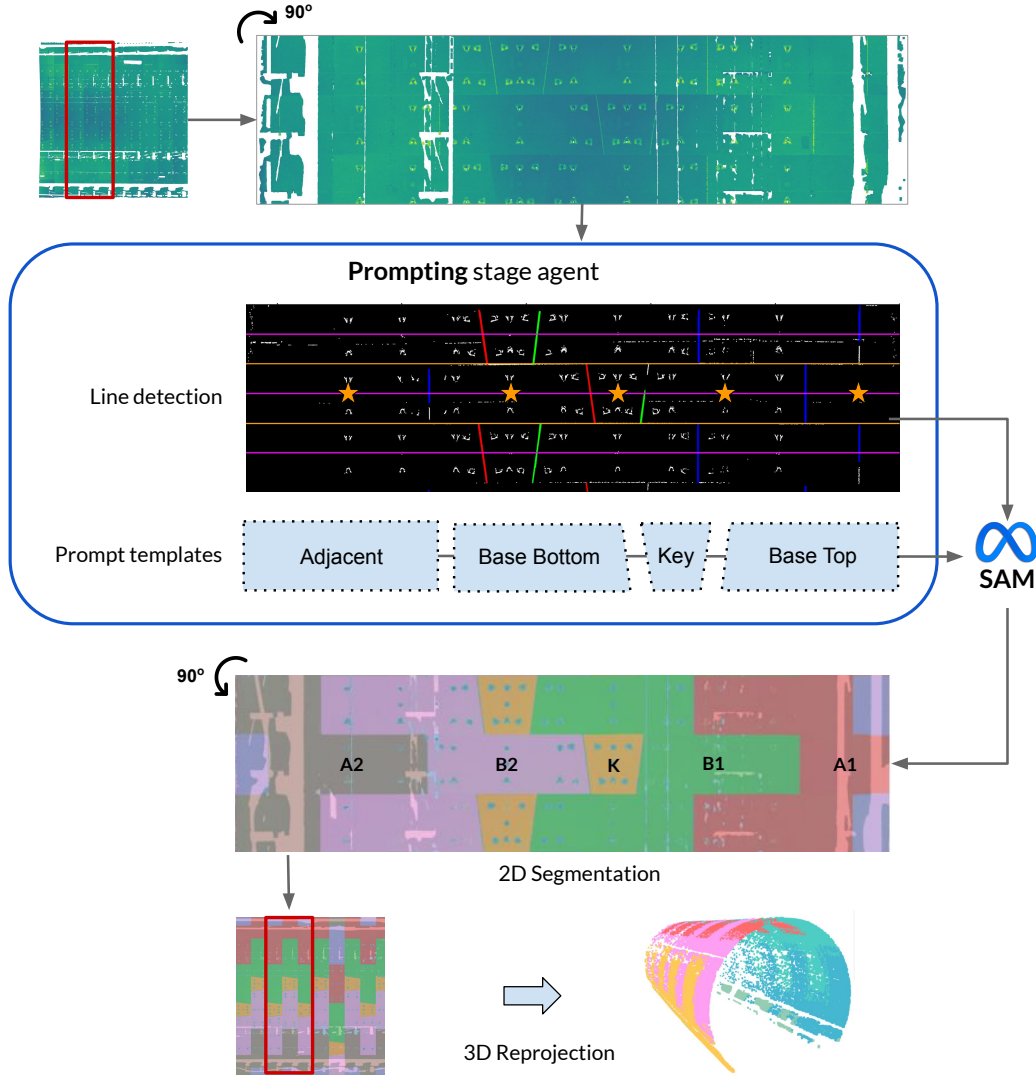
#### 3.3.1. Context design

Each stage agent receives structured context comprising three components: memory, state and knowledge (Fig. 7a). A worked example of the three components for the denoising agent is given in Appendix C.

**Memory** (Appendix C.1) stores the reference tunnel's characteristics (a JSON file containing geometry, point density, coordinate ranges, nearest-neighbour distances) alongside the expert-tuned reference parameters. The agent compares the current tunnel's characteristics against this reference to quantify deviation and adjust its parameters via CoT reasoning (Section 3.3.2). Memory is static: it does not change between stages.

**State** (Appendix C.2) captures cumulative geometric and statistical properties after each pipeline stage executes. These JSON summaries are injected into subsequent agents' prompts, and the state grows as stages execute. Importantly, state alone does not prescribe parameter changes: the numeric summaries of the actual point distribution after upstream processing must be interpreted in the context of parameter semantics and tunnel conditions before they can inform parameter updates.

**Knowledge** (Appendix C.3) supplies domain-specific guidance in human-readable markdown documents. Each stage has its own knowledge document covering three types of guidance: (i) cross-tunnel variation, which describes common lining layouts and structural conditions; (ii) parameter notes, which define each tunable parameter, its empirically validated range, and proven defaults; and (iii) diagnostic rules, which provide constrained guidelines for recommended parameter adjustments. Knowledge is authored once and shared across all tunnels.

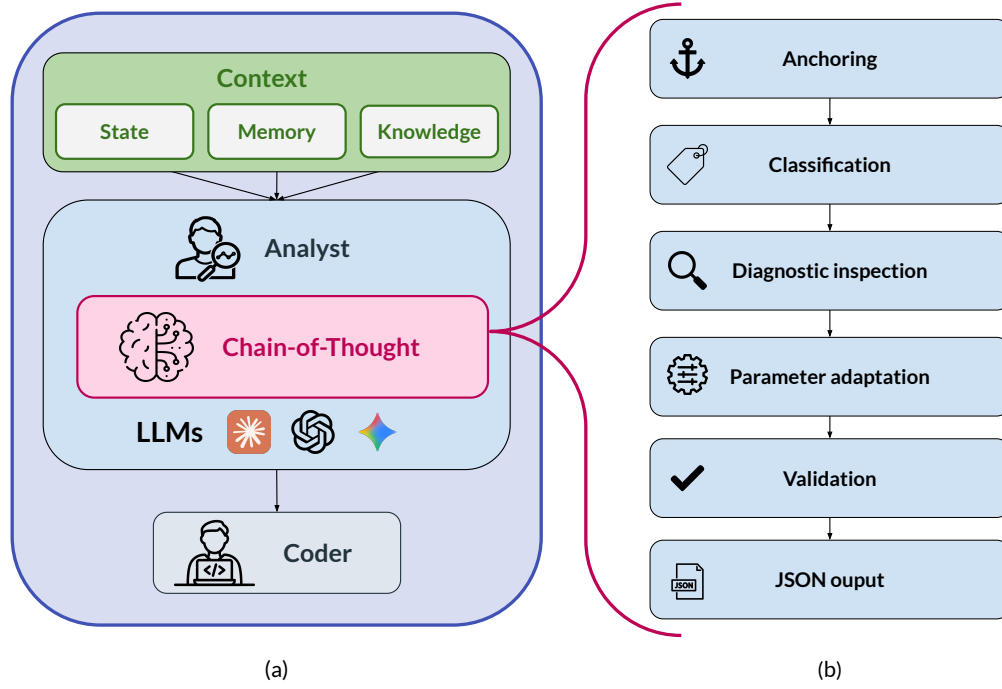


**Figure 6:** Stage 4—Segmenting. The segmenting stage applies Hough-transform line detection [12] to identify ring boundaries, then constructs template-based prompts and feeds them to SAM [22]. SAM produces 2D segment masks that are reprojected into 3D. (Note: rotation is shown for visualisation purposes only.)

### 3.3.2. CoT design

CoT reasoning provides the analytical structure for each agent's decision-making (Fig. 7b). The agent is prompted to follow a five-step protocol (Algorithm 1). A worked example of the full five-step trace is provided in Appendix D.

1. *Referencing:* The agent compares the current tunnel's characteristics against the stored reference to quantify deviation (e.g., +36% diameter increase).
2. *Diagnostic inspection:* The agent attributes the deviation to a plausible cause and identifies which specific tunable parameters are implicated. If signals conflict, the agent is instructed to prioritise *State* over *Memory*, as *State* summarises geometry after upstream processing.
3. *Parameter adaptation:* The agent proposes updates for the implicated parameters. These updates are instructed to stay within the empirical ranges defined in the *Knowledge* component.
4. *Validation:* A self-correction step performs a consistency check to confirm that the proposed values satisfy logical constraints (e.g. a radius lower bound must be smaller than the corresponding upper bound). If a constraint is violated, the value is prompted to move toward to the nearest valid bound. This check-and-correction is executed as an LLM reasoning step, rather than a deterministic software routine, and therefore does not guarantee constraint satisfaction.
5. *JSON output:* The selected parameters and their reasoning trace are packaged into a single schema-conformant JSON object.



**Figure 7:** The left column (a) shows R4Tun stage agent architecture. Each agent maintains a shared and cumulative context that informs an LLM analyst component performing CoT reasoning. The right column (b) shows the five CoT phases: (1) Referencing, (2) Diagnostic inspection, (3) Parameter adaptation, (4) Validation, and (5) JSON output.

**Notation for Algorithm 1.**  $C_{\text{target}}$  denotes the raw characteristics of the current tunnel, while  $M$  stores the reference configuration ( $C_{\text{ref}}, P_{\text{ref}}$ ).  $S$  is the cumulative pipeline state, and  $S_{\text{current}}$  is its most recent summary.  $K$  is a shared knowledge document that encodes parameter semantics, per-parameter admissible bounds  $B$ , and cross-parameter constraints  $R$ . The adaptation procedure computes a geometric deviation descriptor  $\Delta_{\text{geom}}$ , selects an implicated parameter subset  $P_{\text{implicated}}$ , proposes an updated parameter map  $P_{\text{proposed}}$  (indexed as  $P_{\text{proposed}}[p]$ ), and finally returns the schema-formatted JSON object  $P_{\text{adapted}}$ .

### 3.4. Experimental setup

#### 3.4.1. Dataset

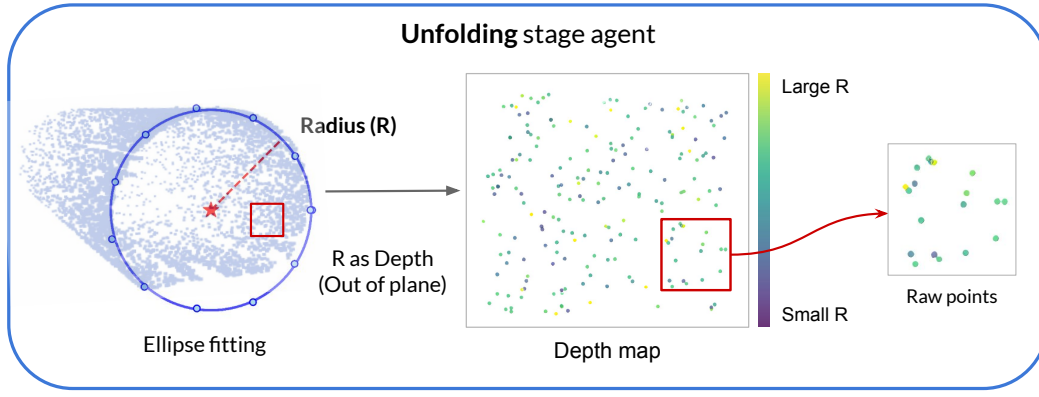
We selected 30 tunnel subsets from the Seg2Tunnel benchmark [26] (Table 2). These subsets cover the main variations in segmental tunnel geometry (diameter, ring length, segment count, key-block layout, and point-density distribution), providing a controlled basis for evaluating adaptation across tunnel categories.

As shown in Fig. 8, staggered and continuous tunnels are grouped as “regular” ( $n = 13$ ): they share the 5.60 m nominal diameter, 1.2 m ring spacing, six segments per ring, and, most importantly, regular patterns of key-block positions similar to the reference. Complex tunnels ( $n = 17$ ) depart from that reference in multiple coupled ways: nominal inner diameter increases by 34% (5.60 m  $\rightarrow$  7.50 m), ring length by 50% (1.2 m  $\rightarrow$  1.8 m), and each ring adds a seventh segment

with an interleaved key-block arrangement that does not repeat ring-to-ring. In addition, off-axis scanning yields non-uniform sampling, partial circumferential coverage, and angle-dependent range and incidence. Together, these differences alter both the geometry visible in the point cloud and the semantic targets the pipeline must recover. Consequently, a fixed parameter set cannot be assumed to transfer from the regular reference; these conditions constitute the primary stress test for whether R4Tun can adapt the same algorithmic pipeline to off-design tunnel conditions. Ground-truth labels are per-point structural class annotations provided by the Seg2Tunnel benchmark; they are used for evaluation only and are not provided to the LLM or pipeline during adaptation.

#### 3.4.2. Experimental design

The experiment follows a cumulative ablation design with four conditions, each adding one context component (Table 3). This design isolates the cumulative contribution of each component while maintaining a controlled comparison. Level 0 (SAM4Tun) is the fixed expert-tuned baseline with no LLM involvement. Levels 1–3 progressively provide the LLM with richer context, enabling the cumulative ablation to attribute improvements to specific components. A separate *Non-LLM* condition (level 0a in Table 3) is included as the Non-LLM adaptive control, allowing the LLMs’ contribution to be read directly from the gap between the *Non-LLM* column and the LLM columns



**Figure 8:** Tunnel lining pattern categories. From left to right: regular (staggered) tunnels exhibit a repeating ring-to-ring pattern; regular (continuous) tunnels maintain a fixed segment order around each ring; complex tunnels show no consistent repeating pattern. **K** denotes the key block, B1–B2 denote base blocks, and A1–A4 denote adjacent blocks around the ring; A4 appears only in the 7-segment complex tunnels.

**Table 2**  
Dataset properties.

| Property           | Regular (T1, T2, T3)             |                         | Complex (T4, T5)       |
|--------------------|----------------------------------|-------------------------|------------------------|
|                    | Staggered (T1, T2)               | Continuous (T3)         |                        |
| Inner diameter     | 5.60 m                           | 5.60 m                  | 7.5 m                  |
| Ring length        | 1.2 m                            | 1.2 m                   | 1.8 m                  |
| Segments/ring      | 6                                | 6                       | 7                      |
| Joint type         | Staggered                        | Continuous              | Complex interleaved    |
| Key-block position | A ring-to-ring repeating pattern | A fixed ring-wise order | No repeating pattern   |
| Scanning           | Single-station                   | Multi-station           | Single-station, offset |
| Eval. schema       | 6-class                          | 6-class                 | 7-class                |
| Count              | 10 subsets                       | 3 subsets               | 17 subsets             |

of Table 5. We codified the per-stage knowledge documents into a deterministic Python rule table as a non-LLM baseline. It maps characterisation fields (e.g., diameter, ring length, segments-per-ring, joint type, point density, and station configuration) to the 18 critical parameters adjusted by the LLMs (Table 8) using explicit “if... then...” rules. The table and script are fixed for all 30 tunnels (regular and complex), with no evaluation-set tuning or subset-specific settings, isolating the value of LLM per-tunnel reasoning. The complete rule specification is provided in Appendix E.

To assess whether adaptation behaviour depends on a specific LLM, each condition was run with three different LLMs (Opus-4.6, GPT-5.4, and Gemini-3-Flash) under identical prompts. Across models, the pipeline code, evaluation scripts, and prompt structure were held constant, while the context content varied by ablation level. Other than setting temperature to 0 to minimise stochasticity (Table 4), all other API settings used vendor defaults. Each of the three ablation conditions was run once per LLM for all 30 tunnels, yielding  $3 \times 3 \times 30 = 270$  pipeline executions plus 30 baseline runs with fixed parameters. Reported results should therefore be interpreted as single-run outcomes under these settings; all three LLMs were accessed

**Table 3**  
Ablation conditions.

| Level | Code    | Condition                  | What the LLM sees                      |
|-------|---------|----------------------------|----------------------------------------|
| 0     | SAM4Tun | Baseline                   | Default parameters                     |
| 0a    | Non-LLM | Rule table                 | 18 parameters set by lookup            |
| 1     | m       | Memory                     | Reference characteristics + parameters |
| 2     | m+s     | Memory+State               | + intermediate pipeline outputs        |
| 3     | m+s+k   | Memory + State + Knowledge | + domain knowledge                     |

via their respective commercial APIs (Table 4). No model-specific prompt tuning was performed. **To probe repeated-call stability without overstating reproducibility, Appendix I reports a second inference pass on nine held-out tunnel-model pairs under the full m+s+k setting.**

Pipeline execution used a single NVIDIA RTX 5060 GPU. Adapted parameter files and CoT reasoning traces were logged for all runs, enabling post-hoc sensitivity analysis (Section 3.4.4).

**Algorithm 1** LLM-driven Parameter Adaptation via CoT

**Require:** Target tunnel raw characteristics  $C_{\text{target}}$   
**Require:** Memory  $M$  (reference characteristics  $C_{\text{ref}}$ , reference parameters  $P_{\text{ref}}$ )  
**Require:** State  $S$  (cumulative pipeline outputs / intermediate summaries)  
**Require:** Knowledge  $K$  (parameter semantics, tunable bounds  $B$ , and cross-parameter constraints  $R$ )  
**Ensure:** Adapted stage parameter JSON  $P_{\text{adapted}}$

- 1: **Phase 1: Referencing**
- 2:  $\Delta_{\text{geom}} \leftarrow \text{Compare}(C_{\text{target}}, C_{\text{ref}})$   $\triangleright$  Quantify deviation from reference
- 3: **Phase 2: Diagnostic inspection**
- 4:  $S_{\text{current}} \leftarrow \text{ExtractLatest}(S)$   $\triangleright$  Most recent stage statistics
- 5: *Reconciliation:* If  $S_{\text{current}}$  conflicts with  $M$ , prioritise  $S_{\text{current}}$
- 6:  $P_{\text{implicated}} \leftarrow \text{IdentifyParameters}(\Delta_{\text{geom}}, S_{\text{current}}, K)$   $\triangleright$  Select parameters relevant to observed deviations
- 7: **Phase 3: Parameter adaptation**
- 8:  $P_{\text{proposed}} \leftarrow \emptyset$
- 9: **for** each parameter  $p \in P_{\text{implicated}}$  **do**
- 10:  $P_{\text{proposed}}[p] \leftarrow \text{Adjust}(p, \Delta_{\text{geom}}, S_{\text{current}}, K)$   $\triangleright$  Propose new value within  $B$  guided by  $K$
- 11: **end for**
- 12: **Phase 4: Validation**
- 13: **for** each parameter  $p \in P_{\text{proposed}}$  **do**
- 14:  $P_{\text{proposed}}[p] \leftarrow \text{SelfValidate}(P_{\text{proposed}}[p], p, K)$   $\triangleright$  Execute an LLM internal reasoning step conditioned on  $K$  (semantics, bounds spec  $B$ , constraints  $R$ ).
- 15: **end for**
- 16: **Phase 5: JSON output**
- 17:  $P_{\text{adapted}} \leftarrow \text{FormatAsJSON}(P_{\text{proposed}})$
- 18: **return**  $P_{\text{adapted}}$

**Table 4**  
LLM configuration.

| Setting          | Value                                  |
|------------------|----------------------------------------|
| Models           | Opus-4.6, GPT-5.4, Gemini-3-Flash      |
| Max tokens       | 16,384                                 |
| Temperature      | 0 (override to minimise stochasticity) |
| Timeout          | 300 s per call                         |
| Prompt format    | Markdown with JSON code                |
| Failure handling | JSON extraction failure                |

**3.4.3. Evaluation metrics**

Segmentation quality is measured primarily by mean Intersection-over-Union (mIoU). For class  $c$ ,

$$\text{IoU}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c}, \quad \text{mIoU} = \frac{1}{C} \sum_{c=1}^C \text{IoU}_c, \quad (1)$$

where  $C$  is the number of segmental blocks (classes) within one ring ( $C = 6$  for regular tunnels;  $C = 7$  for complex tunnels). As a complementary global metric we also report overall accuracy (OA),

$$\text{OA} = \frac{\sum_{c=1}^C \text{TP}_c}{N}, \quad (2)$$

where  $N$  is the total number of points. Because OA can be dominated by majority classes, we treat mIoU as the primary score. We also report per-class IoU to assess whether improvements are broad or concentrated in specific structural classes.

For each condition and LLM, we computed paired per-tunnel improvements as  $\Delta_i = \text{mIoU}_{\text{condition},i} - \text{mIoU}_{\text{baseline},i}$ , with  $n = 13$  for regular tunnels,  $n = 17$  for complex tunnels, and  $n = 30$  overall. We summarise these paired improvements using three standard statistics: (i) two-sided paired  $t$ -test  $p$ -values ( $\alpha = 0.05$ ) for statistical significance, (ii) paired Cohen's  $d$  for effect size, and (iii) bootstrap 95% confidence intervals (CIs) for the mean improvement.

**3.4.4. Sensitivity analysis**

To assess the robustness of adapted parameters, for each parameter, we report the *coefficient of variation* (CV):

$$\text{CV} = \frac{s}{|\bar{v}|}, \quad (3)$$

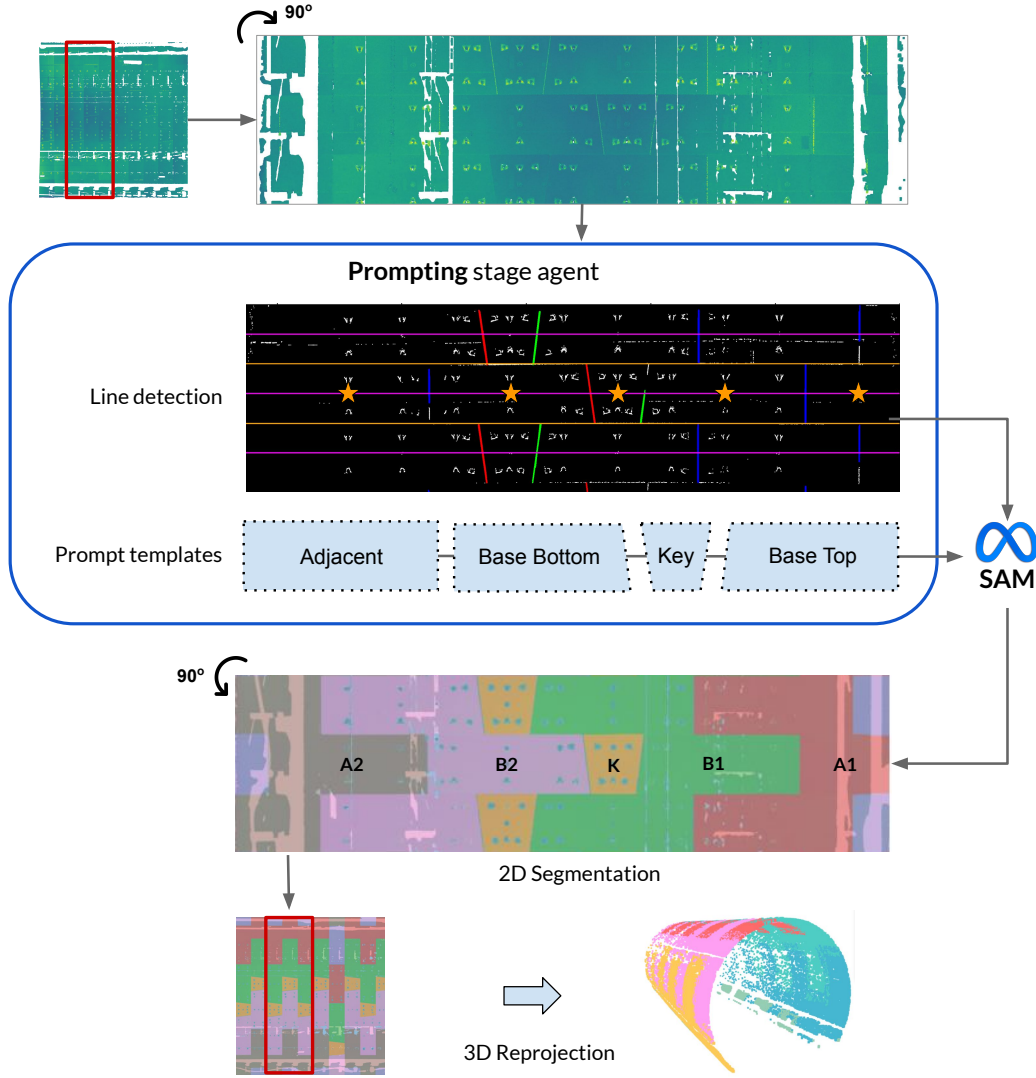
where  $\bar{v} = \frac{1}{N} \sum_{i=1}^N v_i$  and  $s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (v_i - \bar{v})^2}$  are the mean and standard deviation of the adapted values  $\{v_i\}_{i=1}^N$  across  $N = 30$  tunnels. CV measures how much a parameter changes from tunnel to tunnel: (i) a high CV ( $\geq 0.06$ ) identifies parameters that are *tunnel-responsive*; (ii)  $\text{CV} \approx 0$  indicates *baseline corrections* that take the same improved value on every tunnel regardless of geometry. We further examine *cross-LLM consistency*: for parameters that always trigger adaptation, we compare the adapted values, tunnel counts, and tunnel categories produced by each LLM independently.

**4. Results**

This section reports empirical results by comparing overall segmentation performance with the static SAM4Tun baseline and a deterministic non-LLM control, examining how m, s, and k contribute across LLMs, identifying consistently adapted parameters, and analysing the residual errors and runtime trade-off that bound the practical claim.

**4.1. Overall performance**

Table 5 summarises mean mIoU across conditions and LLMs. Under the full m+s+k design, overall mIoU rises from **0.15** (baseline) to **0.32–0.34** across the three



**Figure 9:** Point-cloud visualisation for the tunnels with the largest mIoU gain in each category: regular (staggered; 2-2), regular (continuous; 3-3-1), and complex (5-5). Top (a): SAM4Tun baseline; bottom (b): LLM-adapted pipeline (m+s+k, Opus-4.6). Colours indicate classes; red denotes error; mIoU is shown per example.

LLMs ( $p < 0.0001$ , paired Cohen's  $d = 1.51$ – $1.95$ ); overall OA rises from **0.41** to **0.52–0.55**. The improvement holds across both tunnel categories (Fig. 10). Regular tunnels improve from **0.29** to **0.50–0.54** (paired Cohen's  $d = 1.4$ – $2.2$ ); regular-tunnel OA rises from **0.51** to **0.67–0.71**. Complex tunnels, where the baseline drops to mIoU = **0.04** because several pipeline assumptions no longer match the tunnel conditions, improve to **0.18–0.19** (paired Cohen's  $d = 1.2$ – $2.5$ ); complex-tunnel OA rises from **0.34** to **0.41–0.43**. Fig. 9 shows the largest improvement observed in each category: regular (staggered), regular (continuous), and complex.

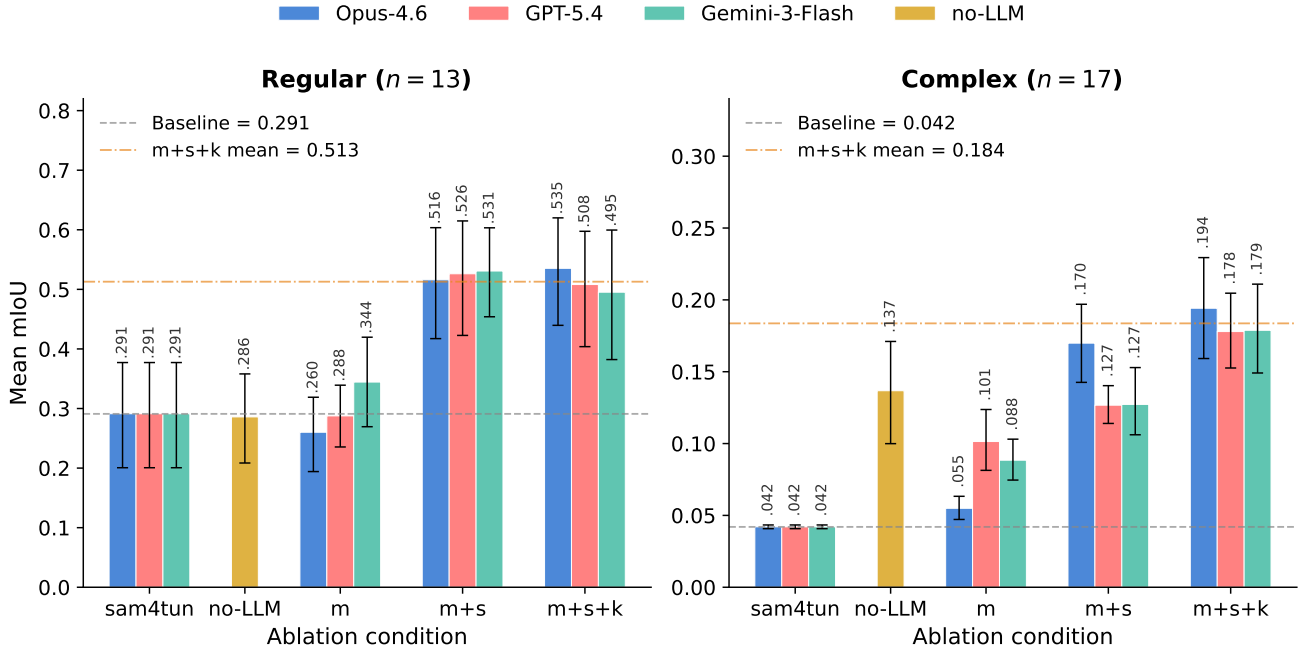
Wall-clock runtime, API-call counts, and per-tunnel input/output token usage with the corresponding indicative USD cost for each LLM under m+s+k are reported in Appendix F (Table 20). Per-class IoU analysis (Appendix G) supports broad improvement across structural classes for regular tunnels and progressive

recovery from near-zero baselines for complex tunnels. The full performance distribution is summarised in Appendix H.

#### Comparison against the non-LLM adaptation.

The non-LLM rule-based adaptation (Table 5, *Non-LLM* column) raises overall mIoU from 0.15 to 0.20 (+0.051,  $p = 0.0035$ ), indicating that part of the gain may reflect deterministic adaptation rather than the LLM-based step alone. The performance distribution varies markedly across tunnel categories: on regular tunnels, the non-LLM adaptation and the static baseline are statistically indistinguishable ( $\approx 0.29$ ;  $p = 0.79$ ), which is likely due to the rule table reproducing the configuration that SAM4Tun was originally tuned for.

On complex tunnels, the rules add +0.095 mIoU ( $p = 0.0001$ ) over the complex category static baseline (0.04) by switching tunnel diameter, ring spacing, segment count, and key-block layout to their large-tunnel



**Figure 10:** Step-wise adaptation results split by tunnel categories. Bars left to right: SAM4Tun (static baseline), *Non-LLM* (rule-based adaptation), *m*, *m+s*, *m+s+k*. The rule baseline closes part of the gap on complex tunnels but stays flat on regular tunnels; LLM (*m+s+k*) is highest in both categories. Error bars are bootstrap 95% CIs.

specification (i.e., lining-design parameters typically available to on-site engineers). Relative to this rule-based baseline, the LLM (*m+s+k*) yields a further increase in mean mIoU from 0.137 to 0.178–0.194 (an additional +0.041 to +0.057), improving 10 of 17 complex tunnels. The remaining seven cases occur primarily in tunnels where the deterministic large-tunnel rules already provide a strong prior.

The gap is largest on the regular category, where the non-LLM rule table remains at the SAM4Tun static baseline (regular-baseline mIoU  $\approx 0.29$ ), whereas the LLM-based adaptation raises mIoU to 0.50–0.54. Under this experimental setup, the additional gain is therefore consistent with an LLM contribution beyond the deterministic control. On the complex category, both methods improve over the static baseline (complex-baseline mIoU = 0.04): the rules reach 0.137, while the LLM further improve mIoU to 0.184. Both the LLM and rule-based adaptations converge toward similarly low absolute mIoU under the single-reference design.

#### 4.2. Ablation analysis

To isolate each context component’s contribution, Table 6 reports cumulative mIoU gains at each ablation step. Fig. 10 visualises the step-wise progression for regular and complex categories across all three LLMs.

**Memory** alone provides an initial reference but is insufficient, and in some settings it degrades performance. On regular tunnels, especially the staggered subsets, memory alone changes mIoU by  $-0.031$  to  $+0.053$  across the three LLMs; Table 6; two of the

three LLMs show small positive averages, while one shows a small negative average). Without intermediate feedback, the agent can propose parameter updates but cannot verify whether they improve or degrade the pipeline outputs.

**State** accounts for the largest observed increment in the ablation study. The *m+s* condition yields the strongest gain relative to baseline across all three LLMs (all  $p < 0.0001$ ; paired Cohen’s  $d = 1.32$ – $1.77$ ), while the memory-only effect is small or inconsistent, indicating that state is the main contributor to the observed improvement. State provides the agent with explicit quantitative evidence of how each stage has transformed the data: radial percentiles for mask bounds, retention rates for denoising aggressiveness, coverage uniformity for upsampling targets. Without state, the agent has only pre-pipeline statistics that may not reflect conditions after unfolding, denoising, or enhancing. The same conclusion holds when the comparator is the Non-LLM baseline rather than the static SAM4Tun: state alone (*m+s*) lifts overall mIoU by 0.10–0.12 above the rule-based adaptation.

One plausible explanation is that the gain associated with state depends on how the model maps raw numeric summaries (percentiles, counts, ratios) to parameter values in light of parameter semantics. We did not test alternative mapping strategies (e.g., regression models); however, the continuous, multidimensional nature of the characteristic space and the non-trivial parameter interactions suggest that capturing this mapping through static rules alone would require

**Table 5**

Overall segmentation results ( $n = 30$ ): mean mIoU and paired  $\Delta$ mIoU vs. baseline with  $p$ -values, effect sizes, and bootstrap 95% CIs.  $\Delta$  is paired against SAM4Tun. OA is reported as a secondary metric, shown as the first line in each condition block.

| Condition          | Statistic          | Non-LLM | Opus-4.6        | GPT-5.4        | Gemini-3-Flash |
|--------------------|--------------------|---------|-----------------|----------------|----------------|
| Baseline (SAM4Tun) | Mean OA            | 0.411   | 0.411           | 0.411          | 0.411          |
|                    | Mean mIoU          | 0.150   | 0.150           | 0.150          | 0.150          |
| Non-LLM            | Mean OA            | 0.407   | —               | —              | —              |
|                    | Mean mIoU          | 0.201   | —               | —              | —              |
|                    | Mean $\Delta$ mIoU | +0.051  | —               | —              | —              |
|                    | $p$ -value         | 0.0035  | —               | —              | —              |
|                    | paired Cohen's $d$ | 0.58    | —               | —              | —              |
|                    | std ( $\Delta$ )   | 0.088   | —               | —              | —              |
| m                  | Mean OA            | —       | 0.397           | 0.428          | 0.443          |
|                    | Mean mIoU          | —       | 0.144           | 0.182          | 0.199          |
|                    | Mean $\Delta$ mIoU | —       | -0.006          | +0.032         | +0.049         |
|                    | $p$ -value         | —       | 0.558           | 0.028          | 0.056          |
|                    | paired Cohen's $d$ | —       | -0.11           | 0.42           | 0.36           |
|                    | 95% CI             | —       | [-0.027, 0.014] | [0.005, 0.058] | [0.006, 0.101] |
| m+s                | Mean OA            | —       | 0.535           | 0.520          | 0.525          |
|                    | Mean mIoU          | —       | 0.320           | 0.299          | 0.302          |
|                    | Mean $\Delta$ mIoU | —       | +0.170          | +0.149         | +0.152         |
|                    | $p$ -value         | —       | < 0.0001        | < 0.0001       | < 0.0001       |
|                    | paired Cohen's $d$ | —       | 1.77            | 1.32           | 1.46           |
|                    | 95% CI             | —       | [0.137, 0.204]  | [0.110, 0.190] | [0.117, 0.189] |
| m+s+k              | Mean OA            | —       | 0.552           | 0.537          | 0.522          |
|                    | <b>Mean mIoU</b>   | —       | <b>0.342</b>    | <b>0.321</b>   | <b>0.316</b>   |
|                    | Mean $\Delta$ mIoU | —       | +0.192          | +0.171         | +0.166         |
|                    | $p$ -value         | —       | < 0.0001        | < 0.0001       | < 0.0001       |
|                    | paired Cohen's $d$ | —       | 1.95            | 1.95           | 1.51           |
|                    | 95% CI             | —       | [0.158, 0.227]  | [0.140, 0.203] | [0.126, 0.204] |

**Table 6**

Cumulative mIoU contribution (mean  $\Delta$  vs. previous level).

| Transition                            | Opus-4.6      | GPT-5.4       | Gemini-3-Flash |
|---------------------------------------|---------------|---------------|----------------|
| Baseline $\rightarrow$ <i>Non-LLM</i> | +0.051        | +0.051        | +0.051         |
| Baseline $\rightarrow$ m              | -0.006        | +0.032        | +0.049         |
| <b>m <math>\rightarrow</math> m+s</b> | <b>+0.176</b> | <b>+0.117</b> | <b>+0.103</b>  |
| m+s $\rightarrow$ m+s+k               | +0.022        | +0.021        | +0.014         |

considerable manual engineering effort per tunnel category.

**Knowledge** provides a small additional gain, concentrated in the complex category. On top of m+s, knowledge raises mean mIoU by +0.014 to +0.022 across the three LLMs. The mean increments are small, but the per-tunnel direction is nevertheless positive. The increment is concentrated where the knowledge document supplies specific tuning guidance that neither memory nor state can reliably infer from numerical summaries alone (e.g., a larger-diameter tunnel often uses wider rings).

**Table 7**

Cross-model summary (m+s+k vs baseline, overall).

| LLM            | Mean $\Delta$ mIoU | 95% CI         | $d$  |
|----------------|--------------------|----------------|------|
| Opus-4.6       | +0.192             | [0.158, 0.227] | 1.95 |
| GPT-5.4        | +0.171             | [0.140, 0.203] | 1.95 |
| Gemini-3-Flash | +0.166             | [0.126, 0.204] | 1.51 |

### 4.3. Cross-model consistency

To assess whether the adaptation behaviour is model-dependent under a fixed prompt and input structure, Table 7 compares the three LLMs on the full m+s+k condition. The three models show similar effect ranges under the full m+s+k condition, although overlapping 95% confidence intervals do not by themselves rule out between-model differences.

### 4.4. Parameter sensitivity

This analysis separates parameters that vary across tunnels from those that remain constant, clarifying which aspects of the adaptation respond to specific tunnel conditions.

Analysis of 270 adapted parameter runs identified 18 parameters that all three LLMs consistently adjust

**Table 8**

Critical parameters identified across all three LLMs (18 total). *Tunnel-responsive* parameters ( $CV \geq 0.06$ ) vary with tunnel geometry; *baseline corrections* ( $CV \approx 0$ ) take near-identical values across tunnels. The control column states what each parameter changes in the SAM4Tun pipeline.

| Stage      | Parameter          | Control                                                      | Behaviour  | Tunnels | CV          | Adapted range / correction  |
|------------|--------------------|--------------------------------------------------------------|------------|---------|-------------|-----------------------------|
| Unfolding  | diameter           | Sets cylindrical unfolding scale.                            | Responsive | 27/30   | 0.072       | [5.31, 7.6]                 |
| Denoising  | mask_r_low         | Sets inner radial gate for retaining lining points.          | Responsive | 30/30   | 0.082       | [2.09, 3.75]                |
| Denoising  | mask_r_high        | Sets outer radial gate for retaining lining points.          | Responsive | 30/30   | 0.147       | [2.78, 4.38]                |
| Denoising  | default_cutoff_z   | Fallback radial cutoff when local density is unreliable.     | Responsive | 29/30   | 0.142       | [2.65, 6.27]                |
| Denoising  | z_step             | Radial histogram bin width for density filtering.            | Responsive | 30/30   | 0.181       | [0.003, 0.005]              |
| Denoising  | smooth_win         | Window length for smoothing radial-density profiles.         | Correction | 30/30   | $\approx 0$ | 3 $\rightarrow$ 5           |
| Denoising  | smooth_offset      | Bias added after smoothing before cutoff selection.          | Correction | 30/30   | $\approx 0$ | -0.003 $\rightarrow$ -0.002 |
| Denoising  | grad_thresh        | Gradient threshold for detecting lining-surface transitions. | Correction | 30/30   | $\approx 0$ | 0.2 $\rightarrow$ 0.15      |
| Denoising  | y_step             | Longitudinal sampling step in density filtering.             | Correction | 30/30   | $\approx 0$ | 0.5 $\rightarrow$ 0.4       |
| Enhancing  | inter_radius       | Neighbour radius used to insert interpolation points.        | Responsive | 30/30   | 0.130       | [0.03, 0.08]                |
| Enhancing  | upsampling_stage1  | First-stage target spacing for progressive upsampling.       | Responsive | 30/30   | 0.064       | [0.055, 0.11]               |
| Enhancing  | curv_thresh        | Curvature cutoff separating smooth panels from joints.       | Correction | 30/30   | $\approx 0$ | 0.0005 $\rightarrow$ 0.005  |
| Enhancing  | depth_low          | Lower depth tolerance for pixel-level interpolation.         | Correction | 30/30   | $\approx 0$ | 0.003 $\rightarrow$ 0.005   |
| Enhancing  | depth_high         | Upper depth tolerance for pixel-level interpolation.         | Correction | 30/30   | $\approx 0$ | 0.008 $\rightarrow$ 0.015   |
| Segmenting | hough_thresh_obliq | Votes required for oblique joint-line detection.             | Responsive | 30/30   | 0.188       | [20, 83]                    |
| Segmenting | hough_thresh_horiz | Votes required for horizontal joint-line detection.          | Responsive | 30/30   | 0.204       | [20, 83]                    |
| Segmenting | hough_thresh_vert  | Votes required for vertical ring-boundary detection.         | Responsive | 28/30   | 0.219       | [320, 980]                  |
| Segmenting | processing.padding | Image padding that controls prompt placement near edges.     | Responsive | 29/30   | 0.265       | [160, 419]                  |

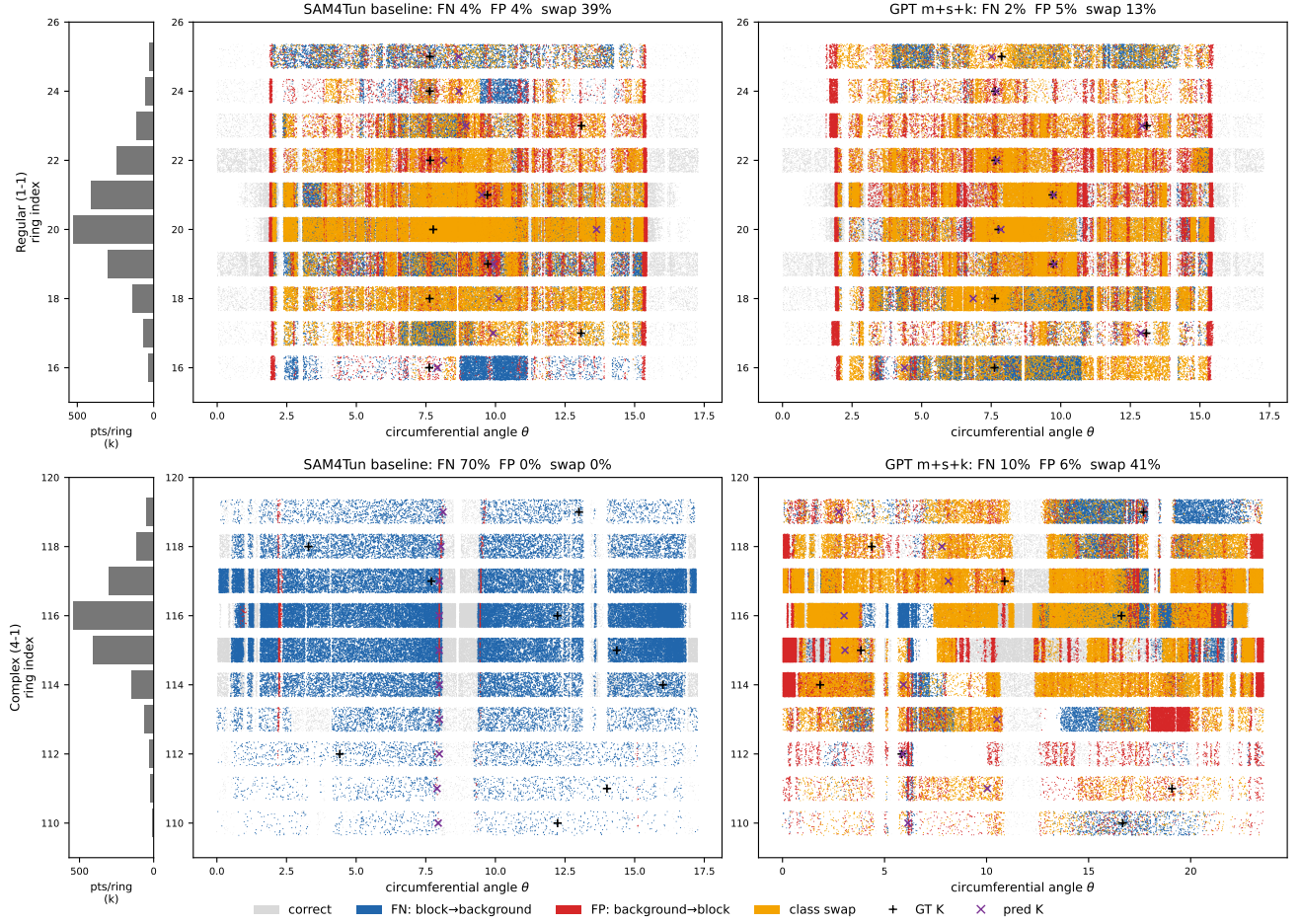
(Table 8). Of these, 11 are tunnel-responsive ( $CV \geq 0.06$ ), in that their adapted values vary with tunnel geometry, clustering by tunnel category. The remaining 7 parameters behave as baseline corrections ( $CV \approx 0$ ), with near-identical values across tunnels, indicating a shared correction trend relative to the SAM4Tun defaults.

#### 4.5. Error analysis

To understand why the absolute metrics stay low and where errors concentrate, each predicted point was assigned to one of four outcomes against the ground truth: correct, false negative (FN, a block point labelled as background), false positive (FP, a background point labelled as a block), or class swap (a block point

assigned to the wrong block type). This decomposition reveals two regimes. On regular tunnels the block boundaries are often found but individual blocks may be assigned the wrong type; on tunnel 1-1, class swaps account for 23% of evaluated points while FN and FP are each about 2%. On complex tunnels the dominant error is under-segmentation; on tunnel 4-1, 69% of block points are labelled as background and the larger blocks are not recovered reliably.

Both regimes trace back to the positional labelling mechanism in the fixed SAM4Tun substrate. Each ring is labelled by first detecting the key block (K), then placing the remaining blocks by stepping a fixed angular template above and below K. This rule is efficient and auditable, but it makes the labels sensitive to four



**Figure 11:** FP/FN error map drawn as a ring-by- $\theta$  raster. Each row is one ring,  $\theta$  is the circumferential angle, and each point is coloured as correct, false negative, false positive, or class swap. The map is intended to show where the remaining errors occur rather than to introduce a new performance metric.

structural bottlenecks: non-uniform point density along a scan, ring-to-ring movement of the K anchor, fixed-offset drift for blocks far from K, and hard-coded walk direction when ring handedness changes.

This analysis clarifies the claim supported by the experiments. R4Tun improves the parameter-sensitive parts of the fixed pipeline, especially Hough thresholds, mask radii, padding, and interpolation settings, but it does not change the positional labelling rule itself. Therefore, the low complex-tunnel mIoU is better interpreted as a ceiling of the single-reference SAM4Tun substrate than as evidence that LLM reasoning cannot adapt parameters. The result supports a bounded adaptation mechanism, not autonomous final inspection.

#### 4.6. Runtime trade-off

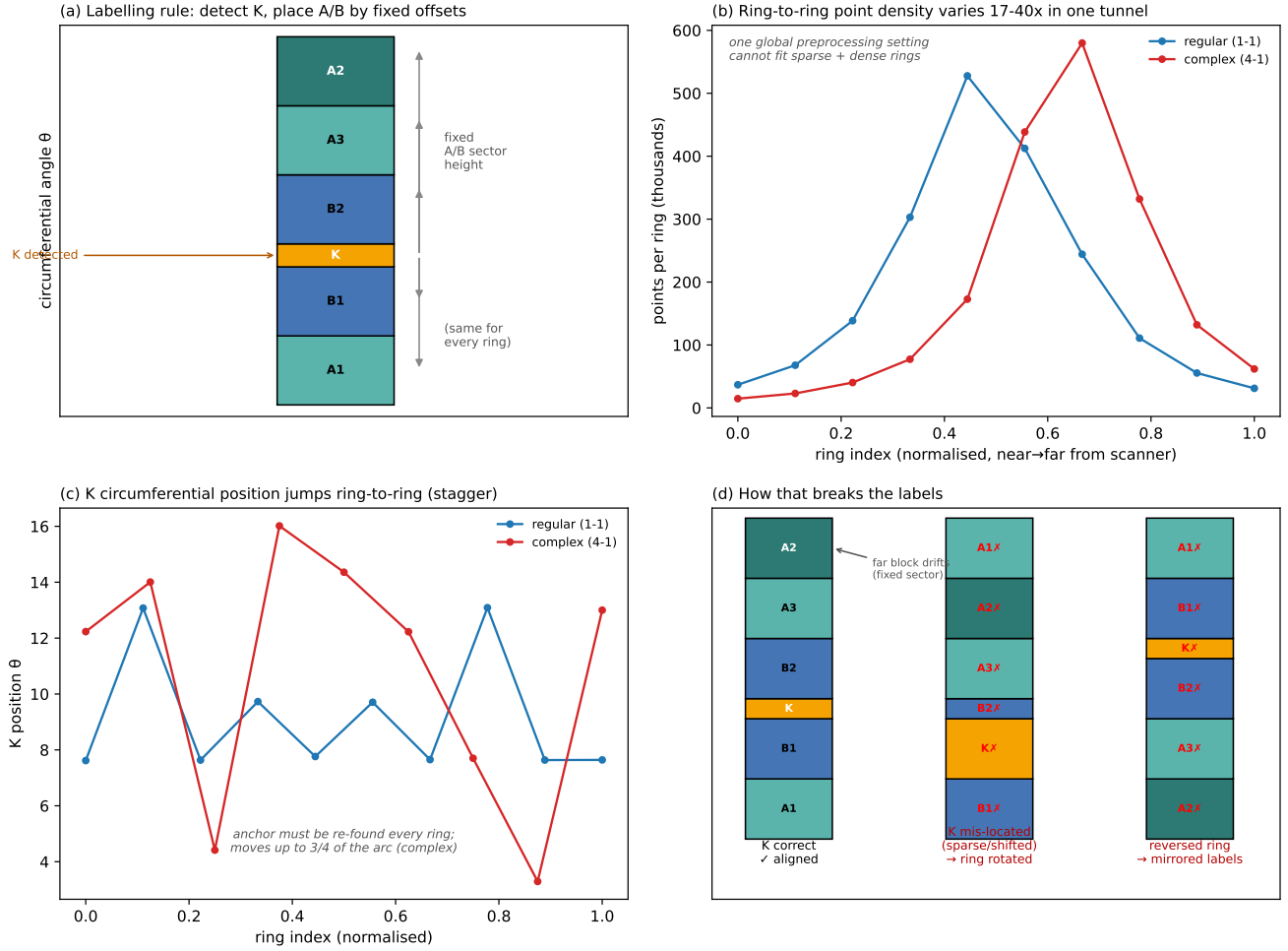
R4Tun trades additional API latency for bounded, logged parameter decisions. In this implementation, the base SAM4Tun processing takes approximately 235 s per tunnel, and the LLM adaptation under m+s+k adds 96–307 s depending on the LLM, for total times of roughly 331–542 s per tunnel across five LLM calls

**Table 9**

Runtime trade-off of the tested R4Tun m+s+k setting. The base SAM4Tun processing time is  $\sim 235$  s per tunnel; extra time is the additional LLM adaptation latency. Timings and API costs are implementation- and vendor-dependent; the table is intended to contextualise overhead, not to define a deployment benchmark.

| LLM            | Extra time             | Total time   | Mean $\Delta$ mIoU |
|----------------|------------------------|--------------|--------------------|
| Gemini-3-Flash | +96 s (0.4 $\times$ )  | $\sim 331$ s | +0.166             |
| Opus-4.6       | +140 s (0.6 $\times$ ) | $\sim 375$ s | +0.192             |
| GPT-5.4        | +307 s (1.3 $\times$ ) | $\sim 542$ s | +0.171             |

(one per pipeline stage; Table 9). The added runtime is therefore not negligible, but it is small compared with collecting labelled training data or retraining a supervised model, and it produces a JSON parameter record and rationale for engineering review. Per-tunnel token usage and indicative API cost (\$0.03–\$1.47 per tunnel across the tested LLMs) are reported in Appendix F.



**Figure 12:** Mechanisms behind class confusion. The fixed pipeline detects K, assigns neighbouring blocks using an angular template, and then reprojects masks to 3D. Density variation, moving K positions, fixed-template drift, and reversed handedness explain why LLM parameter adaptation can improve boundary recovery while still leaving an absolute mIoU ceiling under a single reference.

## 5. Discussion

This section interprets the results in term of R4Tun's intended role: LLM-guided, bounded adaptation of an existing expert-designed pipeline, rather than a general-purpose tunnel segmentation method. It discusses what the results reveal about the m+s+k design, how the single-reference setup limits performance, and the implications for practical deployment and future development.

### 5.1. Key findings

Taken together, the results support a mechanism-level claim rather than a deployment-level one. With the SAM4Tun operators and the expert reference held fixed, structured context improves bounded parameter adaptation across three LLMs (m+s+k: +0.166 to +0.192 mean mIoU, 95% CI lower bounds > 0.126). The practical meaning is that state and knowledge can guide a reviewable parameter update without labelled retraining; the weakening factor is that the resulting

absolute mIoU remains limited, especially on complex tunnels where the fixed positional labelling substrate is poorly matched.

The experimental evaluation yields two primary findings. First, as noted in the ablation study (Section 4.2), the ablation results suggest that the context design improves adaptation across models and tunnel categories, with state providing the main gains (+0.103 to +0.176 mIoU on top of memory). Second, the LLM (m+s+k) gain over the non-LLM rule-based adaptation is concentrated on the regular category, where the rule table cannot improve over the static baseline (Section 4.1). This concentration is consistent with the single-reference design: both comparators start from a configuration tuned on a tunnel that is similar to the regular category. Near the reference, the LLM has more informative context to condition on, and its advantage over deterministic lookup becomes visible; far from the reference (the complex category), neither approach has a matching anchor and the two converge

**Table 10**

Contextual comparison with published tunnel-segmentation and inspection studies. Values are reported to position R4Tun relative to supervised learning and expert-tuned foundation-model pipelines; direct ranking is not intended because inputs, labels, tuning protocol, and evaluation splits differ.

| Study            | Method / task                              | Input                         | Labels                      | Deployment |
|------------------|--------------------------------------------|-------------------------------|-----------------------------|------------|
| Lin et al. [26]  | PointNet++ Level II component segmentation | 3D point cloud                | Yes                         | Learner    |
| Lin et al. [26]  | RandLA-Net Level III scene segmentation    | 3D point cloud                | Yes                         | Learner    |
| Ye et al. [51]   | SAM4Tun component segmentation             | 3D point cloud via 2D prompts | No                          | Expert     |
| Wang et al. [43] | SerialFormer geometric-twin segmentation   | 3D point cloud                | Yes                         | Learner    |
| Yang et al. [50] | YOLOv8-GSD crack segmentation              | Tunnel images                 | Yes                         | Learner    |
| This work        | Static SAM4Tun under one reference         | 3D point cloud                | No                          | None       |
| This work        | R4Tun m+s+k                                | 3D point cloud                | No (GT only for evaluation) | LLM-based  |

toward similarly low absolute performance, with the LLMs retaining a small mean advantage. As reported in Section 4.1, the regular category gap occurs where the deterministic control does not improve over the static baseline, whereas the complex category convergence more likely reflects a shared ceiling than a specific failure of LLM reasoning (Section 5.2).

The comparison in Table 10 explains why the claim is not that R4Tun surpasses trained or expert-tuned segmenters. Supervised Seg2Tunnel models and expert-tuned SAM4Tun achieve much higher absolute mIoU under their own protocols, while R4Tun addresses a different question: whether a single reference-tuned SAM4Tun configuration can be adapted without labels or retraining when reused across off-design tunnel subsets.

In practice, a domain expert calibrates the pipeline on a single reference tunnel, encodes the tuning rationale into structured documents, and deploys R4Tun for subsequent tunnels. Under this single-reference setup, the framework is best suited to targets that remain close to the calibrated reference, and it can also flag when the fixed pipeline begins to fail. The framework supports post-hoc review by logging a rationale alongside each parameter change, and requires no model retraining or labelled domain data. This should be read as a potential workflow implication rather than a validated reduction in expert labour: experts are still required to author the reference configuration, define bounds, inspect rationales, and decide whether low-confidence outputs are acceptable.

The strength of the underlying evidence differs across these claims and should be read accordingly. Confidence is highest for the contribution of state: the m+s effect is large (paired Cohen's  $d = 1.32$ – $1.77$ , all  $p < 0.0001$ ), is replicated independently by three LLMs, and persists when the comparator is the rule-based adaptation rather than the static baseline. Confidence is moderate for the LLM-specific contribution beyond deterministic rules, because that comparison rests on a single codification of the knowledge documents and is concentrated in the regular category; a more elaborate rule table might narrow the gap. Confidence is lowest

for the knowledge increment: the change from m+s to m+s+k (+0.014 to +0.022 mean mIoU) is reported without a separate interval estimate and is comparable in magnitude to the run-to-run variation observed in the repeatability probe (mean  $|\Delta mIoU| = 0.029$  over nine pairs; Appendix I), so we treat it as suggestive rather than established. More generally, all reported values are single-run outcomes at temperature 0; repeated-pass evidence exists only for nine tunnel-model pairs under m+s+k, two of which changed parameters between passes, and the m and m+s levels have no repeated-pass evidence. These constraints bound the confidence of the overall analysis without changing its direction: the cross-LLM agreement and the size of the state effect are unlikely to be artefacts of single-run noise, whereas the smaller increments require replication before stronger claims can be made.

## 5.2. Limitations

R4Tun was evaluated exclusively on the SAM4Tun pipeline and the Seg2Tunnel dataset; as a result, its transferability to other point-cloud processing pipelines and datasets requires further testing. All parameter adaptation is anchored to a single expert-tuned configuration, which creates a low ceiling for both the LLM-guided and the rule-based methods. When no contextual anchor resembles the target tunnel, neither approach has sufficient information to recover strong performance. Expanding R4Tun to encode multiple expert-tuned reference configurations (e.g., one per tunnel category), may further narrow the absolute gap. R4Tun also inherits the assumptions and failure modes of the underlying SAM4Tun operators: when a processing stage fails structurally (e.g., due to severe occlusion, atypical geometry, or acquisition artefacts), LLM reasoning can only re-parameterise that stage rather than replace it. In addition, adaptation quality depends on the completeness and correctness of the offline-authored parameter bounds and knowledge documents; overly conservative bounds or missing cases constrain performance, while overly permissive bounds can increase sensitivity. The m+s+k design is most likely to transfer to other pipelines when four conditions

hold: the pipeline exposes bounded parameters with engineering meaning, at least one expert reference configuration is available, intermediate outputs can be summarised into compact state variables, and the relevant expert knowledge can be encoded in reviewable documents. If any of these conditions is absent, R4Tun becomes a design pattern to test rather than a ready-to-transfer method. The auditability of the generated reasoning traces has not been validated through a formal user study with practising engineers. Finally, parameters are adapted in a single forward pass without iterative self-correction, leaving potential gains from closed-loop refinement unexplored.

## 6. Conclusions

Reliable segmentation of segmental tunnel linings from point clouds remains challenging when tunnel conditions diverge from the expert-tuned reference. This paper presented R4Tun, a multi-agent framework that extends a fixed segmentation pipeline with LLM-guided parameter adaptation. Rather than replacing the underlying geometric operators, R4Tun adapts their parameters by comparing each new tunnel with the reference configuration and by using compact summaries of intermediate pipeline outputs. This design preserves the deterministic structure of the original pipeline while supporting post-hoc review and revision of parameter changes.

In the tested SAM4Tun-Seg2Tunnel setting, the full  $m + s + k$  design raised mean mIoU from 0.150 to 0.316–0.342 across three LLMs, with paired effect sizes of 1.51–1.95 and 95% CI lower bounds above 0.126. The strongest evidence concerns the role of intermediate state: adding state produces the largest ablation increment and remains beneficial when compared with the deterministic rule baseline. The non-LLM control shows that field-observable rules explain part of the improvement, especially on complex tunnels, while the remaining gap on regular tunnels is consistent with an LLM-specific contribution under the same knowledge and parameter bounds.

The practical implication is bounded. R4Tun may help engineers reuse a reference-tuned expert pipeline without labelled retraining by producing parameter JSONs and rationales that can be reviewed, but it does not remove the need for expert-authored references, bounds, knowledge documents, or final engineering judgement. The error analysis shows that low absolute mIoU on complex tunnels is mainly tied to fixed SAM4Tun labelling assumptions and the single-reference setup, so the current evidence supports an adaptation mechanism rather than autonomous final inspection or state-of-the-art segmentation accuracy. The main limitations are inherent to the present design: evaluation is restricted to the SAM4Tun pipeline and the Seg2Tunnel dataset, all adaptation is anchored to

a single expert-tuned reference, the framework inherits the assumptions and failure modes of the underlying operators, adaptation quality depends on the offline-authored bounds and knowledge documents, and parameters are adapted in a single pass without iterative self-correction.

Future work should therefore target the identified ceiling directly: multiple expert reference configurations for different tunnel categories, closed-loop self-evaluation based on intrinsic geometric criteria, and validation on broader tunnel typologies and acquisition conditions. Until those extensions are tested, the defensible conclusion is that structured LLM context can support bounded, reviewable parameter adaptation of a fixed tunnel-segmentation pipeline in the tested setting.

## Data availability

The source code, adapted parameters, and evaluation scripts are available at <https://github.com/Tao-Robominds/R4Tun>. The Seg2Tunnel dataset is publicly available.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Declaration of generative AI use

During this study, the authors evaluated large language models (Opus-4.6, GPT-5.4, and Gemini-3-Flash) as the main experimental systems. The LLMs were also used for language editing and for improving manuscript structure. The authors reviewed and edited all content and take full responsibility for the content of the publication.

## References

- [1] Anthropic, 2025. Effective context engineering for ai agents. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>. Accessed: 2025-12-01.
- [2] Attard, L., Debono, C.J., Valentino, G., Castro, M.D., 2018. Tunnel inspection using photogrammetric techniques and image processing: A review. *ISPRS Journal of Photogrammetry and Remote Sensing* 144, 180–188. doi:10.1016/j.isprsjprs.2018.07.004.
- [3] Bommasani, R., Hudson, D.A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M.S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N., Chen, A., Creel, K., Davis, J.Q., Demszky, D., Donahue, C., Doumbouya, M., Durmus, E., Ermon, S., Etchemendy, J., Ethayarajh, K., Fei-Fei, L., Finn, C., Gale, T., Goel, K., Goodman, N., Grossman, S., et al., 2021. On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258 URL: <https://arxiv.org/abs/2108.07258>.

- [4] Bran, A.M., Cox, S., Schilter, O., Baldassari, C., White, A.D., Schwaller, P., 2024. Chemcrow: Augmenting large-language models with chemistry tools. *Nature Machine Intelligence* 6, 525–535. doi:10.1038/s42256-024-00832-8.
- [5] Camuffo, E., Mari, D., Milani, S., 2022. Recent advancements in learning algorithms for point clouds: an updated overview. *Sensors* 22, 1357. doi:10.3390/s22041357.
- [6] Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., Joulin, A., 2021. Emerging properties in self-supervised vision transformers. *arXiv* doi:10.48550/arXiv.2104.14294, arXiv:2104.14294. version 2, revised 24 May 2021.
- [7] Cha, Y.J., Ali, R., Lewis, J., Buyukozturk, O., 2024. Deep learning-based structural health monitoring. *Automation in Construction* 161, 105324. doi:10.1016/j.autcon.2024.105324.
- [8] Chen, P., Han, B., Zhang, S., 2024. Comm: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving. *arXiv preprint arXiv:2405.00847* URL: <https://arxiv.org/abs/2405.00847>.
- [9] Chua, J., Evans, O., 2025. Are deepseek r1 and other reasoning models more faithful? *arXiv preprint arXiv:2501.07632* URL: <https://arxiv.org/abs/2501.07632>.
- [10] DeepSeek-AI, 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948* URL: <https://arxiv.org/abs/2501.12948>, doi:10.48550/arXiv.2501.12948, arXiv:2501.12948.
- [11] Duan, Y., Qiu, S., Jin, W., Lu, T., Li, X., 2023. High-speed rail tunnel panoramic inspection image recognition technology based on improved yolov5. *Sensors* 23, 6786. doi:10.3390/s23156786.
- [12] Duda, R.O., Hart, P.E., 1972. Use of the hough transformation to detect lines and curves in pictures. *Communications of the ACM* 15, 11–15.
- [13] Ester, M., Kriegel, H.P., Sander, J., Xu, X., 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, 226–231.
- [14] Fischler, M.A., Bolles, R.C., 1981. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM* 24, 381–395.
- [15] Franceschelli, G., Cevenini, C., Musolesi, M., 2025. Training foundation models as data compression: on information, model weights and copyright law. *arXiv preprint arXiv:2501.04023* URL: <https://arxiv.org/abs/2501.04023>.
- [16] Garcia, C.I., Dibattista, M.A., Letelier, T.A., Halloran, H.D., Camelio, J.A., 2024. Framework for llm applications in manufacturing. *Manufacturing Letters* 40, 56–63. doi:10.1016/j.mfglet.2024.08.003.
- [17] Ge, K., Wang, C., Guo, Y., Tang, Y., Hu, Z., Chen, H., 2023. Fine-tuning vision foundation model for crack segmentation in civil infrastructures. *arXiv preprint arXiv:2312.04233* doi:10.48550/arXiv.2312.04233.
- [18] Guo, Z., Zhang, R., Zhu, X., Tang, Y., Ma, X., Han, J., Chen, K., Gao, P., Li, X., Li, H., Heng, P.A., 2023. Point-bind & point-llm: Aligning point cloud with multi-modality for 3d understanding, generation, and instruction following. *arXiv preprint arXiv:2309.00615*.
- [19] Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., Schmidhuber, J., 2024. Metagpt: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352* URL: <https://arxiv.org/abs/2308.00352>.
- [20] Hu, Q., Yang, B., Xie, L., Rosa, S., Guo, Y., Wang, Z., Trigoni, N., Markham, A., 2020. Randla-net: Efficient semantic segmentation of large-scale point clouds. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* URL: <https://arxiv.org/abs/1911.11236>.
- [21] Huang, M.Q., NiniÄĖ, J., Zhang, Q.B., 2021. Bim, machine learning and computer vision techniques in underground construction: current status and future perspectives. *Tunnelling and Underground Space Technology* 108, 103677. doi:10.1016/j.tust.2020.103677.
- [22] Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., Xiao, T., Whitehead, S., Berg, A.C., Lo, W.Y., DollÄĖr, P., Girshick, R., 2023. Segment anything. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 3992–4003.
- [23] Kojima, T., Gu, S.S., Reid, M., Matsuo, Y., Iwasawa, Y., 2023. Large language models are zero-shot reasoners. *arXiv preprint arXiv:2205.11916* URL: <https://arxiv.org/abs/2205.11916>.
- [24] Kolodiazny, M., Vorontsova, A., Konushin, A., Rukhovich, D., 2024. Top-down beats bottom-up in 3d instance segmentation. *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 3566–3574.
- [25] Liang, H.Y., Shen, S.L., Zhou, A., 2026. Eca-enhanced yolo integrated with sahi for multi-defect inspection of tunnel linings. *Tunnelling and Underground Space Technology* 174, 107705. doi:10.1016/j.tust.2026.107705.
- [26] Lin, W., Sheil, B., Zhang, P., Zhou, B., Wang, C., Xie, X., 2024. Seg2tunnel: A hierarchical point cloud dataset and benchmarks for segmentation of segmental tunnel linings. *Tunnelling and Underground Space Technology* 147, 105735. doi:10.1016/j.tust.2024.105735.
- [27] Liu, M., Shi, R., Kuang, K., Zhu, Y., Li, X., Han, S., Cai, H., Porikli, F., Su, H., 2023. Openshape: Scaling up 3d shape representation towards open-world understanding, in: *Advances in Neural Information Processing Systems (NeurIPS)*.
- [28] Mei, L., Yao, J., Ge, Y., Wang, Y., Bi, B., Cai, Y., Liu, J., Li, M., Li, Z.Z., Zhang, D., Zhou, C., Mao, J., Xia, T., Guo, J., Liu, S., 2025. A survey of context engineering for large language models. *arXiv preprint arXiv:2501.04567* URL: <https://arxiv.org/abs/2501.04567>.
- [29] Montero, R., Victores, J.G., MartÄĖñez, S., JardÄĖsn, A., Balaguer, C., 2015. Past, present and future of robotic tunnel inspection. *Automation in Construction* 59, 99–112. doi:10.1016/j.autcon.2015.07.001.
- [30] OpenAI, 2025a. OpenAI o3 and o4-mini system card. <https://openai.com/index/o3-o4-mini-system-card/>. Accessed: 2025-12-01.
- [31] OpenAI, 2025b. Reasoning best practices. <https://platform.openai.com/docs/guides/reasoning-best-practices>. Accessed: 2025-12-01.
- [32] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., Lowe, R., 2022. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155* URL: <https://arxiv.org/abs/2203.02155>.
- [33] Pan, F., Jeon, S., Wang, B., McKenna, F., Yu, S.X., 2024. Zero-shot building attribute extraction from large-scale vision and language models. *arXiv preprint arXiv:2312.12479* URL: <https://arxiv.org/abs/2312.12479>.
- [34] Pauly, M., Gross, M., Kobbelt, L.P., 2002. Efficient simplification of point-sampled surfaces. *Proceedings of IEEE Visualization*, 163–170.
- [35] Qi, C.R., Yi, L., Su, H., Guibas, L.J., 2017. Pointnet++: Deep hierarchical feature learning on point sets in a metric space, in: *Advances in Neural Information Processing Systems (NeurIPS)*. URL: <https://arxiv.org/abs/1706.02413>.

- [36] Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., Sun, M., 2024. Chatdev: Communicative agents for software development. arXiv preprint arXiv:2307.07924 URL: <https://arxiv.org/abs/2307.07924>.
- [37] Ravi, N., Gabeur, V., Hu, Y.T., Hu, R., Ryali, C., Ma, T., Khedr, H., RÄddle, R., Rolland, C., Gustafson, L., Mintun, E., Pan, J., Alwala, K.V., Carion, N., Wu, C.Y., Girshick, R., DollÄr, P., Feichtenhofer, C., 2024. Sam 2: Segment anything in images and videos. arXiv preprint arXiv:2408.00714 URL: <https://arxiv.org/abs/2408.00714>.
- [38] Schult, J., Engemann, F., Hermans, A., Litany, O., Tang, S., Leibe, B., 2023. Mask3d: Mask transformer for 3d semantic instance segmentation. arXiv preprint arXiv:2303.05475 URL: <https://arxiv.org/abs/2303.05475>.
- [39] Sjölander, A., Belloni, V., Ansell, A., Nordström, E., 2023. Towards automated inspections of tunnels: a review of optical inspections and autonomous assessment of concrete tunnel linings. *Sensors* 23, 5457. doi:10.3390/s23125457.
- [40] Strauss, A., Bien, J., Neuner, H., Harmening, C., Seywald, C., Austerreicher, M., Voit, K., Pistone, E., Spyridis, P., Bergmeister, K., 2020. Sensing and monitoring in tunnels: testing and monitoring methods for the assessment of tunnels. *Structural Concrete* 21, 1234–1248. doi:10.1002/suco.201900456.
- [41] Su, C., Hu, Q., Yang, Z., Huo, R., 2024. A review of deep learning applications in tunneling and underground engineering in china. *Applied Sciences* 14, 3234. doi:10.3390/app14083234.
- [42] Wang, B., Chen, Z., Li, M., Wang, Q., Yin, C., Cheng, J.C.P., 2024. Omni-scan2bim: A ready-to-use scan2bim approach based on vision foundation models for mep scenes. *Automation in Construction* 167, 105678. doi:10.1016/j.autcon.2024.105678.
- [43] Wang, Q., Ding, W., Li, F., Qiao, Y., Khoshelham, K., Zhang, F., Wu, Y., 2026. Serialized point cloud segmentation with dilated patchwise attention for generating geometric twins of shield metro tunnels. *Automation in Construction*.
- [44] Wang, Z., Zhu, Z., Wu, Y., Hong, Q., Jiang, D., Fu, J., Xu, S., 2025. Automated tunnel point cloud segmentation and extraction method. *Applied Sciences* 15, 2926. doi:10.3390/app15062926.
- [45] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D., 2023. Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903 URL: <https://arxiv.org/abs/2201.11903>.
- [46] Weidner, L., Walton, G., 2024. Generalized extraction of bolts, mesh, and rock in tunnel point clouds: a critical comparison of geometric feature-based methods using random forest and neural networks. *Remote Sensing* 16, 678. doi:10.3390/rs16040678.
- [47] Xiang, V., Snell, C., Gandhi, K., Albalak, A., Singh, A., Blagden, C., Phung, D., Rafailov, R., Lile, N., Mahan, D., Castrioto, L., Fränken, J.P., Haber, N., Finn, C., 2025. Towards System 2 reasoning in LLMs: Learning how to think with meta chain-of-thought. arXiv preprint arXiv:2501.04682 URL: <https://arxiv.org/abs/2501.04682>, doi:10.48550/arXiv.2501.04682, arXiv:2501.04682.
- [48] Xu, P., Ping, W., Wu, X., McAfee, L., Zhu, C., Liu, Z., Subramanian, S., Bakhturina, E., Shoyebi, M., Catanzaro, B., 2024a. Retrieval meets long context large language models. arXiv preprint arXiv:2310.03025 URL: <https://arxiv.org/abs/2310.03025>.
- [49] Xu, R., Wang, X., Wang, T., Chen, Y., Pang, J., Lin, D., 2024b. Pointllm: Empowering large language models to understand point clouds, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 131–147.
- [50] Yang, K., Bao, Y., Li, J., Fan, T., Tang, C., 2024. Deep learning-based yolo for crack segmentation and measurement in metro tunnels. *Automation in Construction* 168, 105818. doi:10.1016/j.autcon.2024.105818.
- [51] Ye, Z., Lin, W., Faramarzi, A., Xie, X., NiniÄĖ, J., 2025. Sam4tun: No-training model for tunnel lining point cloud component segmentation. *Tunnelling and Underground Space Technology* 158, 106401. doi:10.1016/j.tust.2025.106401.
- [52] Yue, Y., Liu, H., DonÄ, M., Liu, X., Saler, E., Cui, J., da Porto, F., 2026a. Dual-backbone fusion network for damage segmentation in cultural heritage buildings. *Automation in Construction* 182, 106769. doi:10.1016/j.autcon.2026.106769.
- [53] Yue, Y., Liu, H., Lin, C., Meng, X., Liu, C., Zhang, X., Cui, J., Du, Y., 2024a. Automatic recognition of defects behind railway tunnel linings in GPR images using transfer learning. *Measurement* 224, 113903. doi:10.1016/j.measurement.2023.113903.
- [54] Yue, Y., Liu, H., Liu, X., et al., 2026b. Rapid non-contact road surface inspection using tire-road coupling. *Advanced Engineering Informatics* 74, 104774. doi:10.1016/j.aei.2026.104774.
- [55] Yue, Y., Zhang, S., Liu, H., Hong, F., Liu, C., Cui, J., Du, Y., 2024b. Damage mechanism of a shield tunnel with cavities behind the concrete lining: An insight from a scaled model test. *Tunnelling and Underground Space Technology* 153, 105998. doi:10.1016/j.tust.2024.105998.
- [56] Zhang, Y., Sun, R., Chen, Y., Pfister, T., Zhang, R., Arik, S.Ä., 2024. Chain of agents: Large language models collaborating on long-context tasks. arXiv preprint arXiv:2406.02818 URL: <https://arxiv.org/abs/2406.02818>.
- [57] Zhang, Z., Yao, Y., Zhang, A., Tang, X., Ma, X., He, Z., Wang, Y., Gerstein, M., Wang, R., Liu, G., Zhao, H., 2023. Igniting language intelligence: the hitchhiker's guide from chain-of-thought reasoning to language agents. arXiv preprint arXiv:2311.11797 URL: <https://arxiv.org/abs/2311.11797>.
- [58] Zou, X., Yang, J., Zhang, H., Li, F., Li, L., Wang, J., Wang, L., Gao, J., Lee, Y.J., 2023. Segment everything everywhere all at once. arXiv preprint arXiv:2304.06718 URL: <https://arxiv.org/abs/2304.06718>.

## Appendices

### A. Baseline parameter tables

Tables 11–14 report the SAM4Tun baseline parameter values used as the reference configuration for all adaptation experiments.

### B. Characteriser fields

Table 15 summarises the characteriser fields used to describe each tunnel and to populate the per-stage state provided to the agents.

### C. Context components: denoising agent example

Appendices C and D share a single worked example: the *denoising* agent applied to tunnel 4-1 (m+s+k condition, Opus 4.6). This appendix reproduces the three context components (memory, state, knowledge) that the agent receives as input; Appendix D reproduces the CoT trace and adapted-parameter JSON it returns.

#### C.1. Memory: reference vs target raw characteristics

Memory pairs the reference tunnel’s raw characteristics with those of the target tunnel using an identical schema, so the agent can compute deviations field-by-field (Table 16).

Memory also bundles the SAM4Tun reference parameters for the stage (Table 12), so the agent always has a known-good baseline to deviate from.

#### C.2. State: cumulative outputs of upstream stages

State grows as the pipeline executes. For the denoising agent, state consists of the unfolded characteristics produced by the preceding unfolding stage, supplied as a reference/target pair (Table 17).

#### C.3. Knowledge: parameter semantics, ranges, and constraints

Knowledge is a stage-specific Markdown document, authored once and shared across all tunnels. It enumerates each parameter together with its empirically validated range, defaults, and inter-parameter constraints. The denoising knowledge document is reproduced verbatim below.

**Tunable parameters.** *mask\_r\_low* (m), inner radial gate before depth histogramming, range [2.09, 3.75] (baseline 2.7); *mask\_r\_high* (m), outer radial gate, range [2.78, 4.38] (baseline 2.8); *default\_cutoff\_z* (m), fallback radial cut off when a  $\theta$ -bin lacks reliable counts, range [2.65, 6.27] (baseline 2.7); *z\_step* (m), radial bin width per histogram column, range [0.003, 0.005] (baseline 0.001).

**Table 11**

Unfolding parameters (sam4tun baseline).

| Parameter                     | Value |
|-------------------------------|-------|
| <i>delta</i>                  | 0.005 |
| <i>slice_spacing_factor</i>   | 1.2   |
| <i>vertical_filter_window</i> | 4.5   |
| <i>ransac_threshold</i>       | 1     |
| <i>ransac_probability</i>     | 0.9   |
| <i>ransac_inlier_ratio</i>    | 0.75  |
| <i>ransac_sample_size</i>     | 5     |
| <i>polynomial_degree</i>      | 3     |
| <i>num_samples_factor</i>     | 1210  |
| <i>diameter</i>               | 5.5   |

**Table 12**

Denoising parameters (sam4tun baseline).

| Parameter                    | Value  |
|------------------------------|--------|
| <i>mask_r_low</i>            | 2.7    |
| <i>mask_r_high</i>           | 2.8    |
| <i>y_step</i>                | 0.5    |
| <i>z_step</i>                | 0.001  |
| <i>grad_threshold</i>        | 0.2    |
| <i>smoothing_window_size</i> | 3      |
| <i>smoothing_offset</i>      | −0.003 |
| <i>default_cutoff_z</i>      | 2.7    |

**Table 13**

Enhancing parameters (sam4tun baseline).

| Parameter                        | Value  |
|----------------------------------|--------|
| <i>upsamp_stage1_target_dist</i> | 0.08   |
| <i>upsamp_stage2_target_dist</i> | 0.04   |
| <i>upsamp_stage3_target_dist</i> | 0.02   |
| <i>curvature_threshold</i>       | 0.0005 |
| <i>depth_threshold_low</i>       | 0.003  |
| <i>depth_threshold_high</i>      | 0.01   |
| <i>inter_radius</i>              | 0.06   |
| <i>duplicate_threshold</i>       | 0.02   |
| <i>num_neighbors</i>             | 20     |
| <i>num_interpolations</i>        | 2      |
| <i>resolution</i>                | 0.005  |
| <i>window_size</i>               | 9      |

#### Proven defaults (fixed, not searched).

*smoothing\_window\_size* = 5, *smoothing\_offset* = −0.002, *grad\_threshold* = 0.15, *y\_step* = 0.4.

**Locked parameters.** All other keys in the stage JSON remain at the pipeline defaults reported in Table 12.

**Constraints.** *mask\_r\_low* < *mask\_r\_high*; *default\_cutoff\_z* should stay coherent with the upstream unfolding diameter (order of magnitude  $\approx$  diameter/2, with diameter in metres).

The four other stages (unfolding, enhancing, detecting, segmenting) follow the same four-block template;

**Table 14**

Detecting parameters (sam4tun baseline).

| Parameter             | Value    |
|-----------------------|----------|
| binary_threshold      | 127      |
| morph_kernel_size     | [3, 3]   |
| dilation_iterations   | 1        |
| hough_thresh_oblique  | 50       |
| minLineLength_oblique | 100      |
| maxLineGap_oblique    | 40       |
| hough_thresh_horiz    | 50       |
| minLineLength_horiz   | 100      |
| maxLineGap_horiz      | 10       |
| hough_thresh_vert     | 500      |
| angle_range_obliq_pos | [6, 9]   |
| angle_range_obliq_neg | [-9, -6] |
| merge_distance        | 3        |
| ring_spacing_constant | 1.2      |
| resolution            | 0.005    |

**Table 15**

Characteriser fields by stage. Raw fields are available to all stages; each subsequent group becomes available after the corresponding stage completes.

| Source   | Fields                                                                                                                                                                 |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Raw      | estimated diameter, tunnel length, tunnel height<br>z-range (min, max)<br>mean / median / min nearest-neighbour distance                                               |
| Unfolded | r-percentiles ( $p_{10}$ , $p_{99}$ ), h-span, $\theta$ -span, $\theta$ -range<br>median / std nearest-neighbour distance<br>intensity median, intensity min           |
| Denoised | mean / median / std nearest-neighbour distance<br>estimated diameter, tunnel length, surface completeness<br>surface regularity, average curvature, section curvatures |
| Enhanced | total points, median / mean nearest-neighbour distance<br>coverage uniformity<br>template spacing suitability, current median spacing                                  |

the per-stage knowledge documents are released in the project's code repository.

## D. Worked CoT trace

The following trace continues the worked example from Appendix C: the denoising agent receives the memory, state, and knowledge inputs above, executes the five-step CoT protocol (Section 3.3.2), and emits a single schema-conformant JSON object that the denoising stage consumes unchanged.

**Table 16**

Memory excerpt (raw characteristics). Reference is the expert-tuned tunnel; target is tunnel 4-1.

| Field                               | Reference | Target (4-1) | $\Delta$ |
|-------------------------------------|-----------|--------------|----------|
| Estimated diameter (m)              | 5.50      | 7.50         | +36%     |
| Tunnel length (m)                   | 12.16     | —            | —        |
| Tunnel height (m)                   | 5.08      | —            | —        |
| Mean NN distance (m)                | 0.0082    | 0.0114       | +39%     |
| Median NN distance (m)              | 0.0065    | 0.0089       | +37%     |
| Point density (pts/m <sup>3</sup> ) | 2,466     | 1,842        | -25%     |

**Table 17**

State excerpt (unfolded characteristics, after the unfolding stage).

| Field                     | Reference | Target (4-1) |
|---------------------------|-----------|--------------|
| r-percentile $p_{10}$ (m) | 2.30      | 2.63         |
| r-percentile $p_{99}$ (m) | 2.77      | 4.02         |
| h-span (m)                | 12.08     | 6.08         |
| $\theta$ -span (rad)      | 17.28     | 7.65         |
| Median NN distance (m)    | 0.0042    | 0.0086       |

*Step 1 — Referencing.* Diameter is 7.5 m vs reference 5.5 m (+36%). Unfolded r-percentiles are  $p_{10} = 2.63$ ,  $p_{99} = 4.02$ , vs reference  $p_{10} = 2.30$ ,  $p_{99} = 2.77$ . Median nearest-neighbour distance is 0.0086 m vs reference 0.0042 m, indicating point density at  $\sim 50\%$  of the reference.

*Step 2 — Diagnostic inspection.* Large-diameter, moderate-sparse target. The reference mask\_r\_low/mask\_r\_high (2.7/2.8) bracket only the reference  $p_{99}$  and would clip  $\sim 60\%$  of wall points at the target radius; the radial gate must widen. default\_cutoff\_z (2.7) is below the target  $p_{99}$  (4.02) and would force the fallback path to discard valid wall points; it must rise. Sparse density requires z\_step to relax from 0.001 so each radial bin retains enough samples for the gradient test.

*Step 3 — Parameter adaptation.* Set mask\_r\_low = 2.37 (just below target  $p_{10}$ ); mask\_r\_high = 3.80 (above target  $p_{99} = 4.02$  would exceed the empirical range [2.78, 4.38] when widened further; 3.80 is the largest value that retains  $\geq 99\%$  of wall points while staying within range); default\_cutoff\_z = 3.7 (above mask\_r\_high and order  $\approx$  diameter/2); z\_step = 0.005 (proven default for sparse scans). All other parameters take the proven defaults from the knowledge block.

*Step 4 — Validation.* mask\_r\_low (2.37) < mask\_r\_high (3.80); both lie within their empirical ranges. default\_cutoff\_z (3.7) is consistent with the unfolding diameter (7.5 m, half-diameter 3.75 m). z\_step (0.005) lies in [0.003, 0.005]. All keys present, all types match Table 12.

*Step 5 — JSON output.*

**Table 18**

Worked example output (denoising agent, tunnel 4-1, m+s+k, Opus 4.6).

| Parameter             | Baseline | Adapted | Rationale                   |
|-----------------------|----------|---------|-----------------------------|
| mask_r_low            | 2.7      | 2.37    | Just below target $p_{10}$  |
| mask_r_high           | 2.8      | 3.80    | Covers target $p_{90}$      |
| default_cutoff_z      | 2.7      | 3.7     | $\approx \text{diameter}/2$ |
| z_step                | 0.001    | 0.005   | Sparse-density              |
| smoothing_window_size | 3        | 5       | Proven default              |
| smoothing_offset      | -0.003   | -0.002  | Proven default              |
| grad_threshold        | 0.2      | 0.15    | Proven default              |
| y_step                | 0.5      | 0.4     | Proven default              |

**Table 19**

On-site rules baseline: inputs and rule-driven parameters.

| Stage      | Parameter             | Rule                               |
|------------|-----------------------|------------------------------------|
| Unfolding  | diameter              | $= d_{\text{measured}}$            |
| Denoising  | mask_r_low            | $= d/2 - 0.15$                     |
| Denoising  | mask_r_high           | $= d/2 + 0.15$                     |
| Denoising  | default_cutoff_z      | $= \text{mask\_r\_high} + 0.05$    |
| Segmenting | ring_spacing_constant | $= L_{\text{ring}}$ (1.2 or 1.8 m) |
| Segmenting | segment_per_ring      | $= 6$ (T1–T3) or $7$ (T4–T5)       |

```
{
  "mask_r_low": 2.37,
  "mask_r_high": 3.80,
  "y_step": 0.4,
  "z_step": 0.005,
  "grad_threshold": 0.15,
  "smoothing_window_size": 5,
  "smoothing_offset": -0.002,
  "default_cutoff_z": 3.7
}
```

Table 18 summarises the parameters that changed and the rationale tied to the trace above.

The same parameter keys are adapted by GPT-5.4 and Gemini 3 Flash with values within  $\pm 5\%$  of the Opus outputs for this tunnel.

## E. On-site rules baseline specification

The rules baseline uses only field-observable inputs and adjusts five parameters from the sam4tun defaults. All other parameters retain their expert-tuned values. Inputs per tunnel: nominal diameter  $d$  (design drawings), tunnel family (visual inspection), ring length  $L_{\text{ring}}$  (drawings), and segment count (counted visually). For T4–T5 tunnels, SAM template dimensions (segment\_width, K\_height, AB\_height) are scaled by  $d/5.5$  and  $L_{\text{ring}}/1.2$  from the 5.5 m reference. No characteriser output, pipeline intermediate state, or domain-knowledge document is used.

**Table 20**

Runtime, API calls, and per-tunnel API cost. Token counts are as reported by each vendor API; USD figures use vendor list prices at submission and are indicative.

| Metric                                                          | Value                          |
|-----------------------------------------------------------------|--------------------------------|
| LLM API calls per tunnel                                        | 5 (one per pipeline stage)     |
| Total API calls (full ablation)                                 | 150 per condition per LLM      |
| Wall-clock time per tunnel                                      | $\sim 24$ min                  |
| GPU                                                             | Single NVIDIA RTX 5060         |
| Retraining required                                             | None                           |
| Labelled data required                                          | None (GT for evaluation only)  |
| <i>Per-tunnel tokens and cost (m+s+k, mean over 30 tunnels)</i> |                                |
| Claude Opus 4.6                                                 | 64,854 in / 6,599 out, \$1.47  |
| GPT-5.4 (high-effort reasoning)                                 | 64,414 in / 35,263 out, \$0.43 |
| Gemini 3 Flash                                                  | 83,658 in / 3,058 out, \$0.03  |

**Table 21**

Per-class IoU—Regular tunnels ( $n = 13$ , 6-class, Opus 4.6).

| Class      | sam4tun | rules | memory | m+s   | m+s+k |
|------------|---------|-------|--------|-------|-------|
| Background | 0.640   | 0.597 | 0.559  | 0.741 | 0.751 |
| K-block    | 0.192   | 0.222 | 0.129  | 0.373 | 0.386 |
| B1-block   | 0.261   | 0.250 | 0.206  | 0.527 | 0.540 |
| A1-block   | 0.253   | 0.263 | 0.276  | 0.537 | 0.542 |
| A2-block   | 0.150   | 0.144 | 0.159  | 0.380 | 0.420 |
| A3-block   | 0.286   | 0.289 | 0.259  | 0.519 | 0.550 |
| B2-block   | 0.256   | 0.236 | 0.232  | 0.534 | 0.558 |

## F. Runtime, API calls, and cost

Table 20 reports the compute setup and, for one m+s+k run, the per-tunnel input/output token counts and indicative USD cost for each LLM (averaged over the 30 tunnels, five stage calls per tunnel).

Pricing assumed (per  $10^6$  tokens): Opus 4.x \$15 in / \$75 out; GPT-5.x \$1.25 in / \$10 out; Gemini 3-Flash \$0.30 in / \$2.50 out. The three models receive the same Markdown prompt at every stage; the spread in input-token counts reflects tokeniser density (Gemini counts more tokens on the JSON-heavy prompt), and GPT-5.4's larger output reflects internal reasoning tokens billed at the output rate with `reasoning.effort=high`.

## G. Per-class IoU breakdown

Tables 21 and 22 report per-class IoU for Opus 4.6 (representative; other LLMs show the same pattern) alongside the rules baseline. For regular tunnels, rules achieve per-class IoU comparable to sam4tun while all LLM conditions improve roughly uniformly. For complex tunnels (rules  $n = 17$  including 3 failed tunnels scored as zero), rules recover segment structure from near-zero for most classes but not K-block; the LLM conditions achieve further improvement on regular tunnels. B2-block remains the hardest class, requiring the 7-segment layout guidance from the knowledge component.

**Table 22**

Per-class IoU—Complex tunnels ( $n = 17$ , 7-class, Opus 4.6).  
Rules include 3 failed tunnels scored as zero.

| Class      | sam4tun | rules | memory | m+s   | m+s+k |
|------------|---------|-------|--------|-------|-------|
| Background | 0.337   | 0.534 | 0.358  | 0.513 | 0.520 |
| K-block    | 0.000   | 0.000 | 0.034  | 0.183 | 0.159 |
| B1-block   | 0.000   | 0.041 | 0.001  | 0.084 | 0.135 |
| A1-block   | 0.000   | 0.072 | 0.005  | 0.128 | 0.155 |
| A2-block   | 0.000   | 0.083 | 0.006  | 0.143 | 0.116 |
| A3-block   | 0.000   | 0.149 | 0.017  | 0.092 | 0.119 |
| A4-block   | 0.000   | 0.116 | 0.019  | 0.100 | 0.104 |
| B2-block   | 0.000   | 0.099 | 0.000  | 0.000 | 0.043 |

**Table 23**

Performance distribution (mean across 3 LLMs).

| Metric    | sam4tun | rules | memory | m+s   | m+s+k |
|-----------|---------|-------|--------|-------|-------|
| Mean mIoU | 0.150   | 0.201 | 0.175  | 0.304 | 0.320 |
| Std       | 0.166   | 0.132 | 0.136  | 0.228 | 0.218 |
| Min       | 0.032   | 0.000 | 0.042  | 0.082 | 0.072 |
| Max       | 0.532   | 0.484 | 0.471  | 0.682 | 0.679 |

## H. Performance distribution

Table 23 summarises the distribution of mIoU across tunnels (reported as the mean across the three LLMs for each condition).

## I. LLM inference repeatability

Under m+s+k with temperature set to 0, we compared the primary adapted parameter JSONs (run 1) to a second LLM inference pass (run 2) on the same tunnel prompts. Run 1 parameters were left on disk before run 2 so the orchestrator could skip pipeline stages whose outputs matched run 1. Scripts: `bootstrap_repeatability_run1.py`, `run_repeatability.py`, and `reproducibility_analysis.py` -layout repeatability (repository root). Outputs are logged under `logs/{tunnel}/repeatability/`.

Table 24 summarises the first nine tunnel-model pairs recovered from held-out reruns on complex tunnels 4-4, 5-3, and 5-4 (three LLMs each). The full protocol targets 90 pairs (30 tunnels  $\times$  3 LLMs); aggregate statistics below update when the remaining pairs complete.

On these nine pairs, median critical-parameter identity was 100% (mean 88%) and mean  $|\Delta\text{mIoU}|$  was 0.029, below the paired adaptation gain versus the static baseline ( $\Delta\text{mIoU} \approx 0.17\text{--}0.19$ ). Large mIoU swings on tunnel 4-4 Opus ( $0.245 \rightarrow 0.047$ ) coincide with lower parameter identity and reflect failure-recovery reruns rather than benign stochasticity; the full 90-pair batch uses a single fresh run 2 per combo.

**Table 24**

LLM repeatability under m+s+k (temperature 0): run 1 vs run 2 on nine held-out complex-tunnel pairs. *Crit. identity* is the fraction of the 18 critical parameters (Table 8) taking identical values in both runs.

| Tunnel                             | LLM            | mIoU run1 | mIoU run2 | Crit. identity |
|------------------------------------|----------------|-----------|-----------|----------------|
| 4-4                                | Opus-4.6       | 0.245     | 0.047     | 13/18 (72%)    |
| 4-4                                | GPT-5.4        | 0.133     | 0.075     | 8/18 (44%)     |
| 4-4                                | Gemini-3-Flash | 0.108     | 0.108     | 18/18 (100%)   |
| 5-3                                | Opus-4.6       | 0.178     | 0.178     | 18/18 (100%)   |
| 5-3                                | GPT-5.4        | 0.124     | 0.124     | 18/18 (100%)   |
| 5-3                                | Gemini-3-Flash | 0.143     | 0.143     | 18/18 (100%)   |
| 5-4                                | Opus-4.6       | 0.207     | 0.207     | 18/18 (100%)   |
| 5-4                                | GPT-5.4        | 0.098     | 0.098     | 18/18 (100%)   |
| 5-4                                | Gemini-3-Flash | 0.122     | 0.118     | 13/18 (72%)    |
| Median critical-parameter identity |                |           |           | 100%           |
| Mean $ \Delta\text{mIoU} $         |                |           |           | 0.029          |